

DNS

Names, and the machine that resolves them

The Domain Name System translates domain names into IP addresses. This lecture covers what it stores, how that data is distributed, how a single lookup works, and how a new domain is published.

Yaxiong Xie



You have never typed the address of a website

① **chatgpt.com**

and every other service you use, the same way

google.com

youtube.com

amazon.com

github.com

buffalo.edu

the machine you reach is identified by a 32-bit number
and you have never seen it

Every service you use is reached by a name.

• — Why this matters

A name people remember is worth paying for

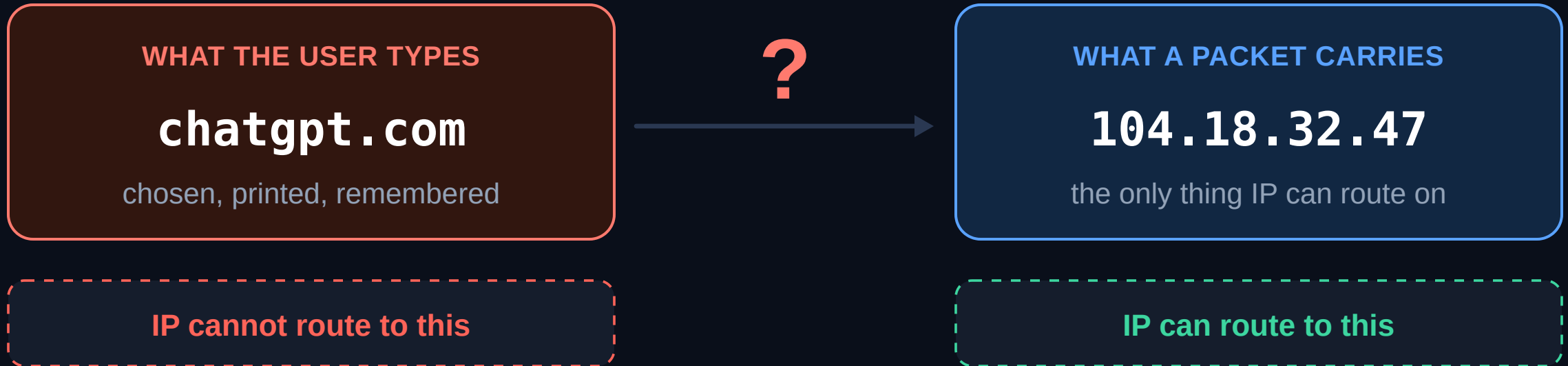
ai.com	\$70,000,000	2025
voice.com	\$30,000,000	2019
360.com	\$17,000,000	2015
chat.com	\$15,500,000	2023 · now OpenAI's
NFTs.com	\$15,000,000	2022
rocket.com	\$14,000,000	2024

a domain sale transfers a name and nothing else — no hardware changes hands

These are the highest publicly reported prices paid for a single name.

- — Why this matters

A packet can only be addressed with a number



something has to turn the left side into the right side

before a single packet leaves your machine

That translation is the Domain Name System.

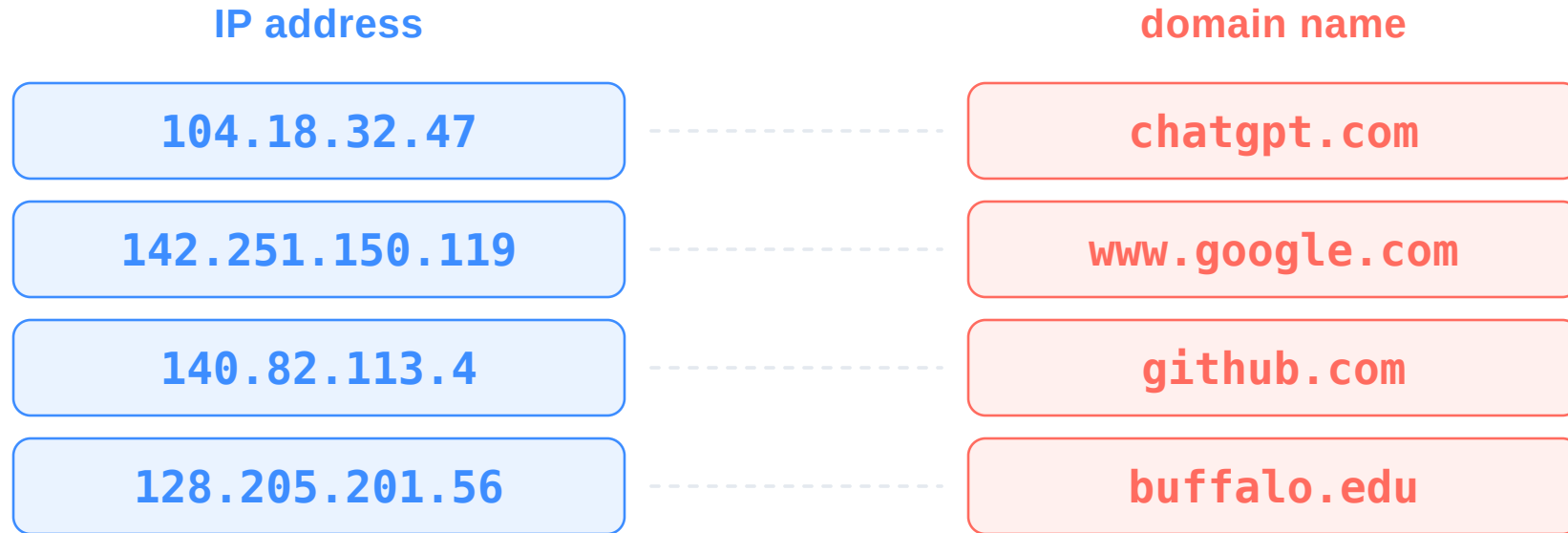
Why do we need domain names at all?

01

Why names exist

Being easy to remember is the obvious reason, and a shared text file would satisfy it. Five more reasons follow, and the last of them is what forces a distributed protocol.

Memorability is the obvious reason, and the weakest one



the same four services you just saw, identified two ways

Everything on the last three slides is explained by this one reason, and it is a real one. But it only asks for a lookup table with two columns. It does not ask for a distributed protocol, and it is the first of six.

So why is DNS not just a shared file? Five more reasons follow.

1 Why names

2 One server

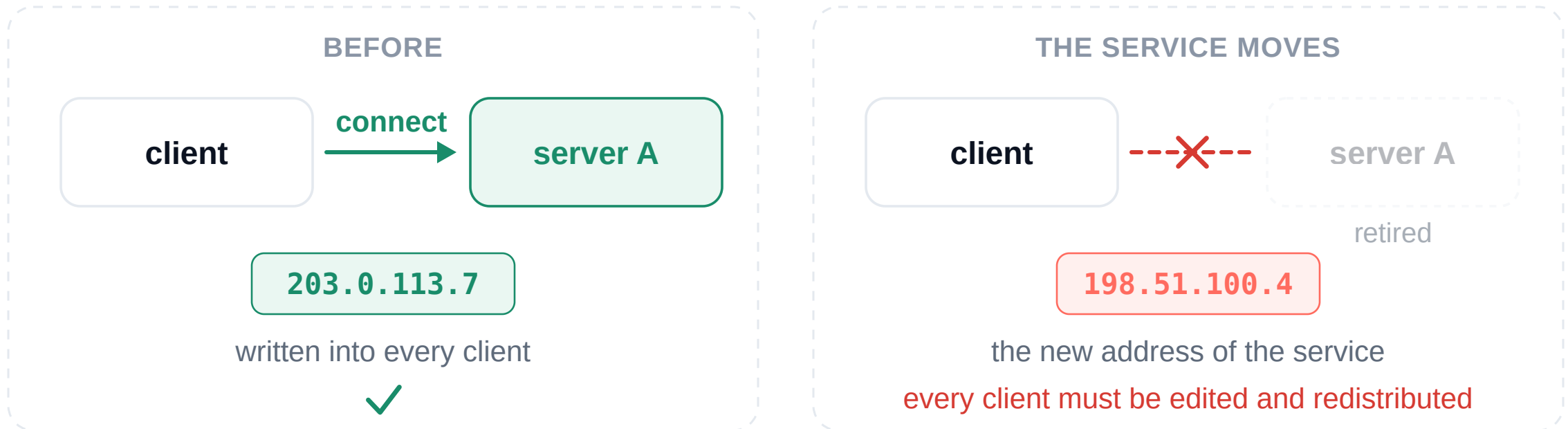
3 The tree

4 The lookup

5 Caching

6 Publishing

An IP address identifies a location, and locations change



An IP address describes where a machine currently sits in the routing system. It changes when the service moves to different hardware, a different provider or a different network. Clients should not have to be updated when that happens.

The name stays constant while the address is allowed to change.

1 Why names

2 One server

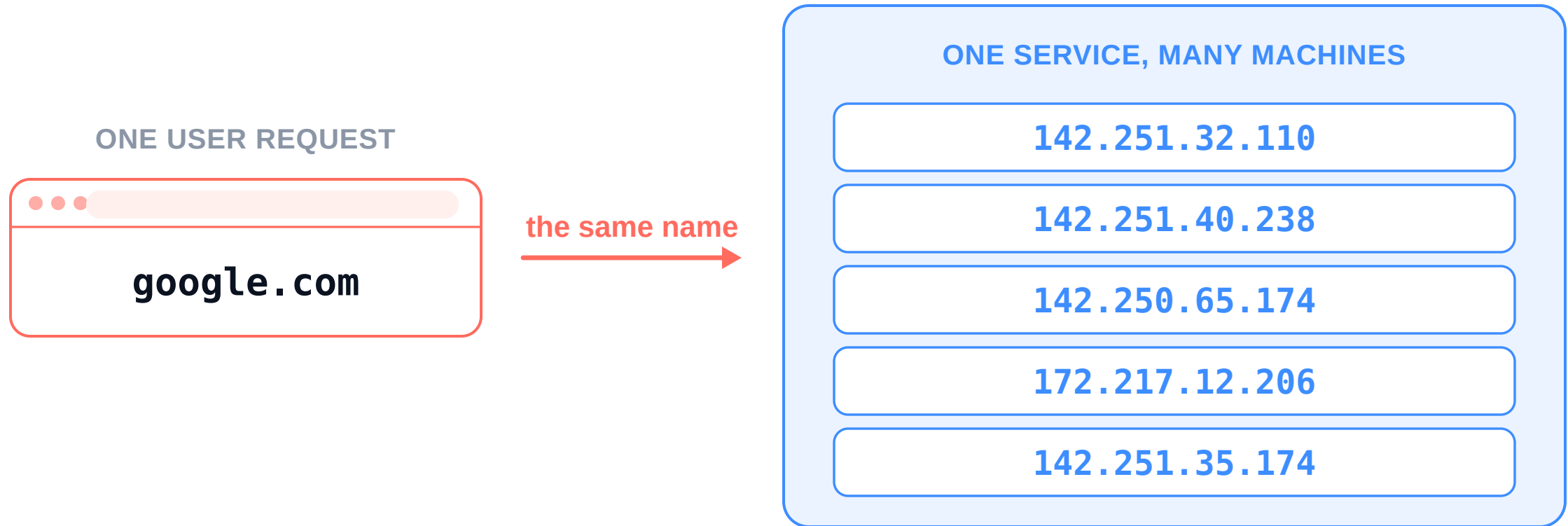
3 The tree

4 The lookup

5 Caching

6 Publishing

One service can run on many machines



any one of these is a correct answer to that request

One name must be able to map to many addresses.

1 Why names

2 One server

3 The tree

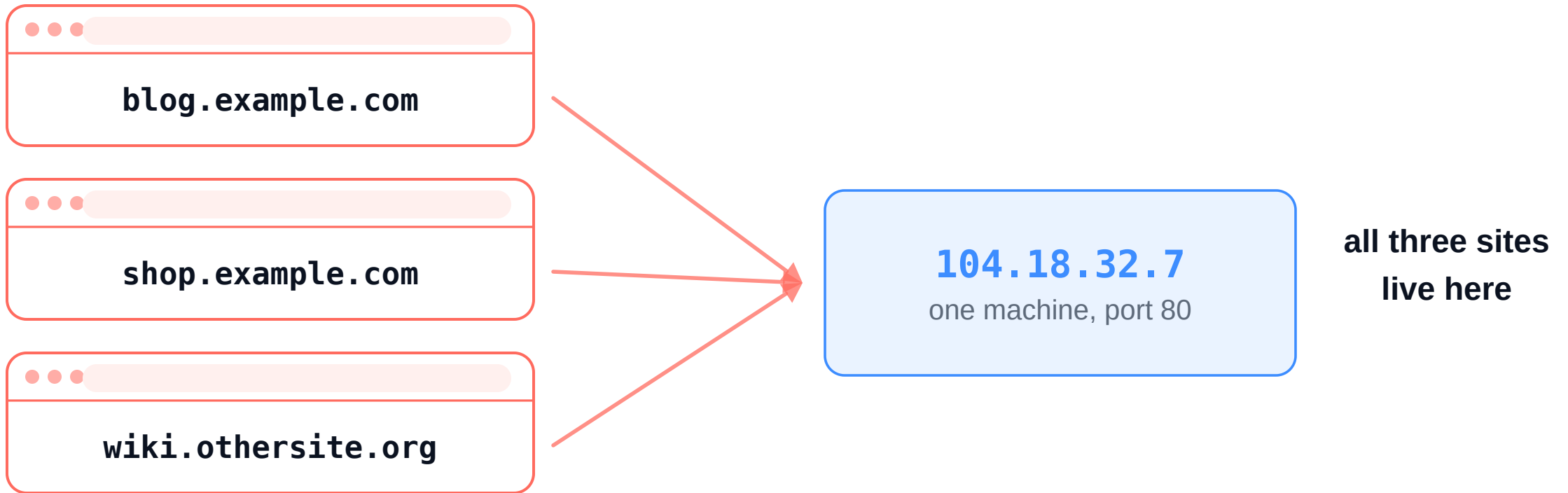
4 The lookup

5 Caching

6 Publishing

One machine can serve many different names

THREE USER REQUESTS



The mapping between names and addresses is many-to-many.

1 Why names

2 One server

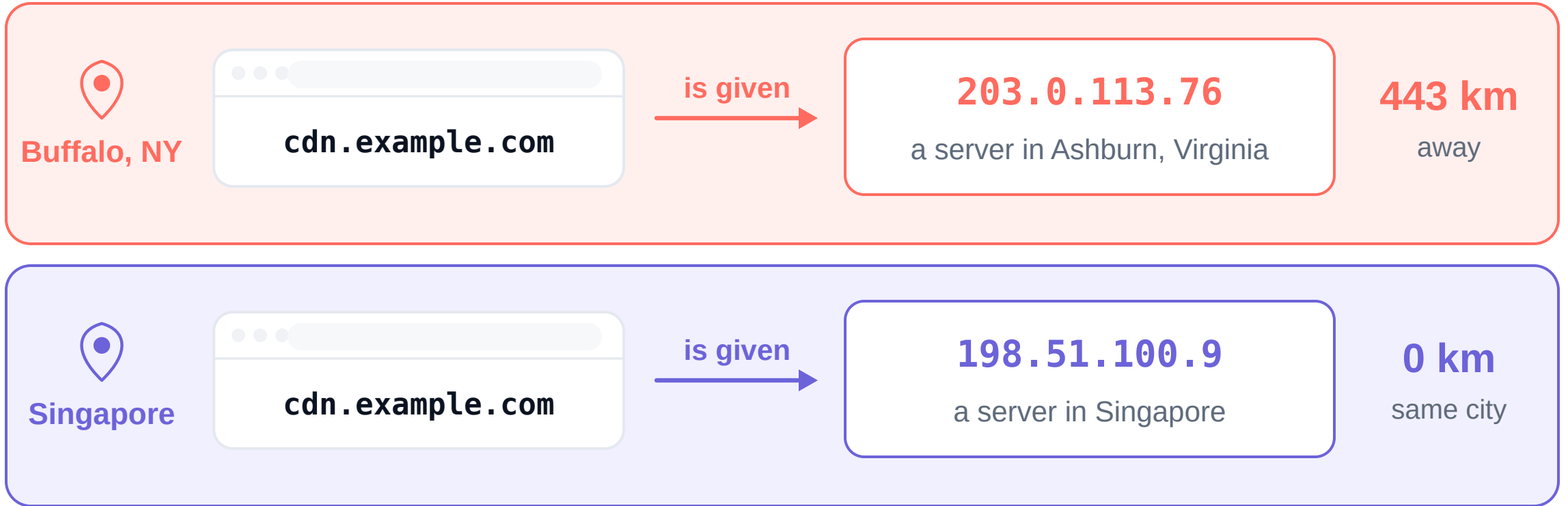
3 The tree

4 The lookup

5 Caching

6 Publishing

The best address depends on where the client is



had Singapore been given the Ashburn address, every packet would travel 15,500 km

The same name, asked from two places, has two correct answers.

1 Why names

2 One server

3 The tree

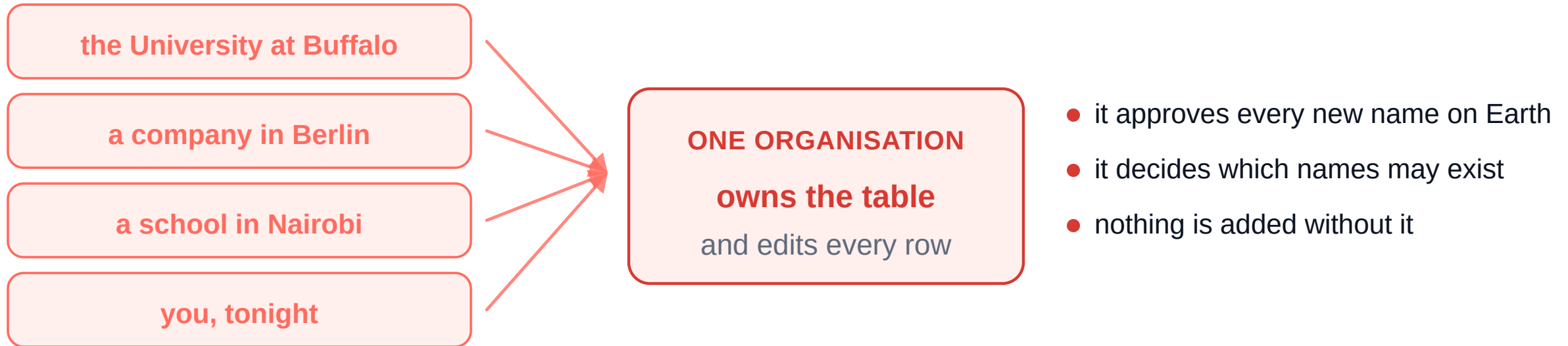
4 The lookup

5 Caching

6 Publishing

With one table, one organisation decides every name

you want to create `lab7.cse.buffalo.edu`



no amount of copying changes who holds the pen

One file means one gatekeeper for the whole Internet.

1 Why names

2 One server

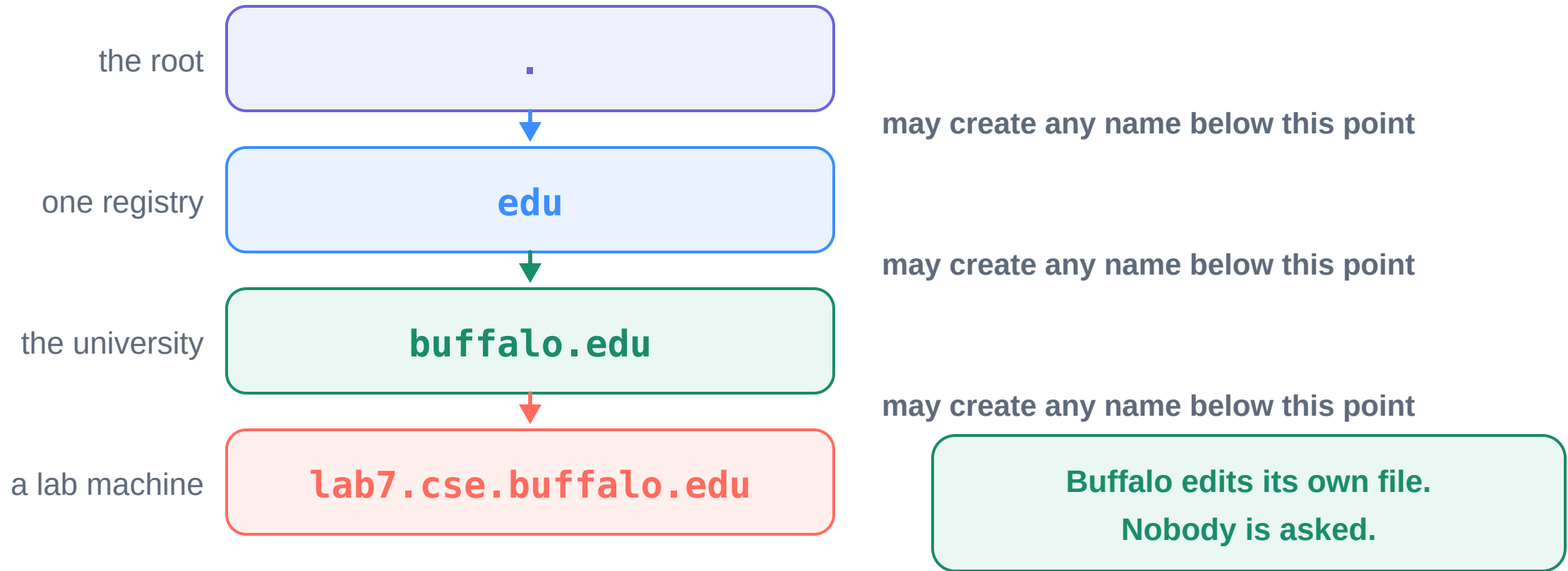
3 The tree

4 The lookup

5 Caching

6 Publishing

Delegation hands naming authority down the tree



Finding which of those files holds a name is what needs a protocol.

1 Why names

2 One server

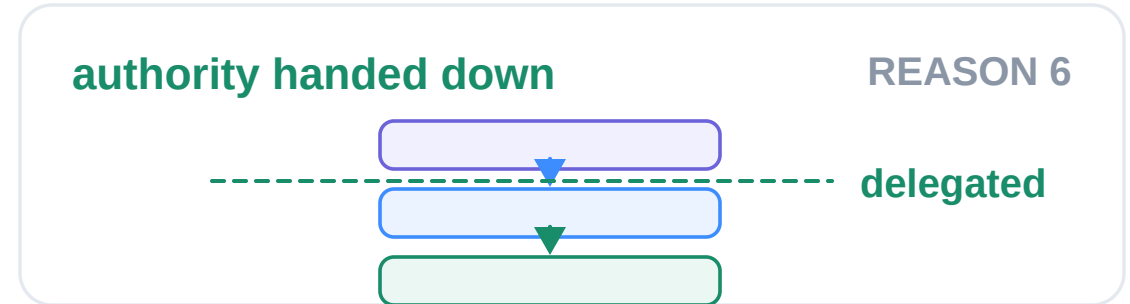
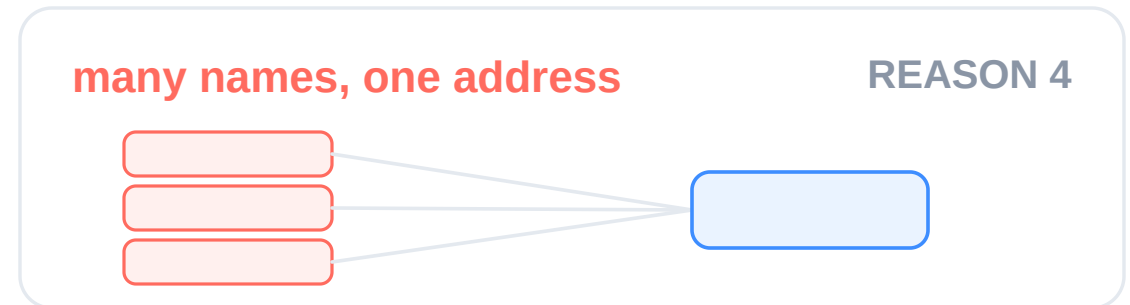
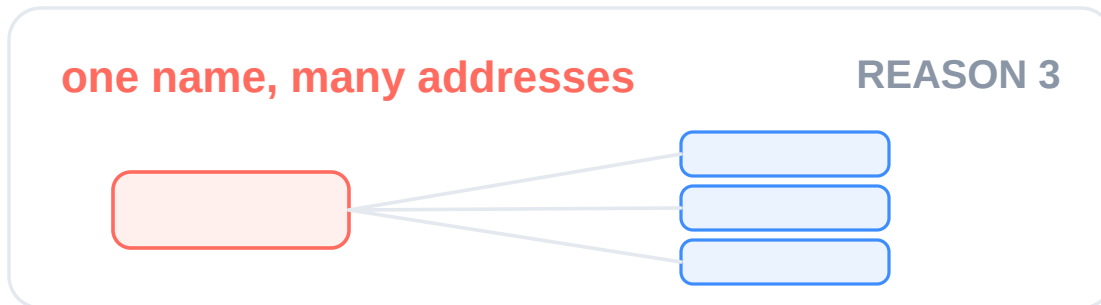
3 The tree

4 The lookup

5 Caching

6 Publishing

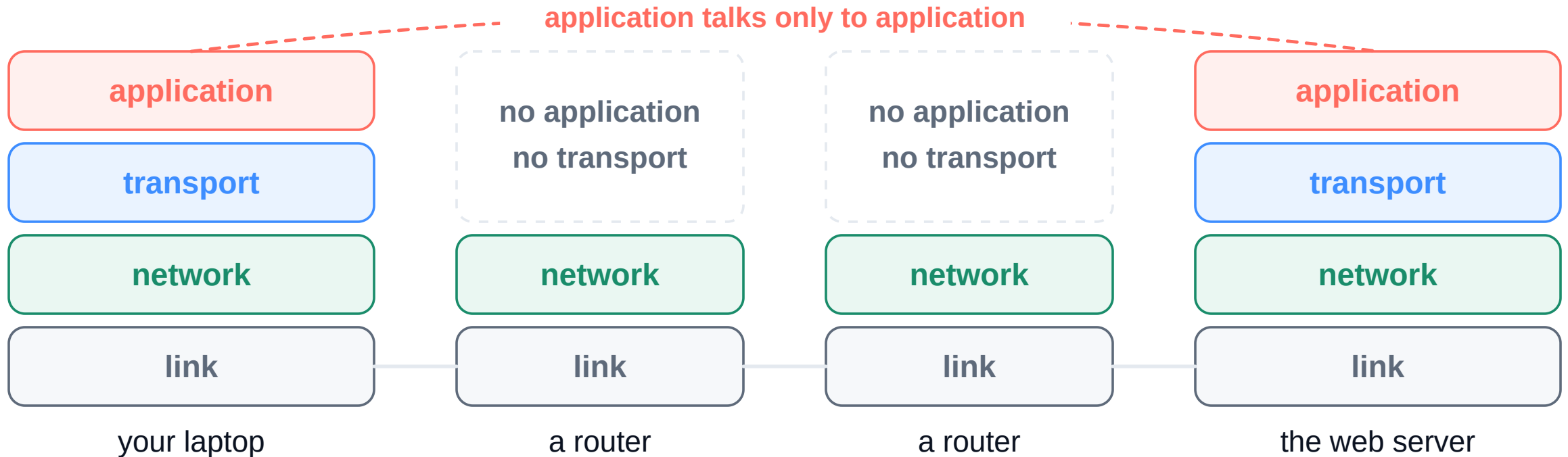
All six reasons ask for one thing: a mapping you can look up



DNS is that mapping, and these four properties are its job description.

- 1 Why names
- 2 One server
- 3 The tree
- 4 The lookup
- 5 Caching
- 6 Publishing

Only the end points run the application layer



a router in between reads the IP header and forwards; nothing above it

Above IP, only the two ends do anything. The network in between does not.

1 Why names

2 One server

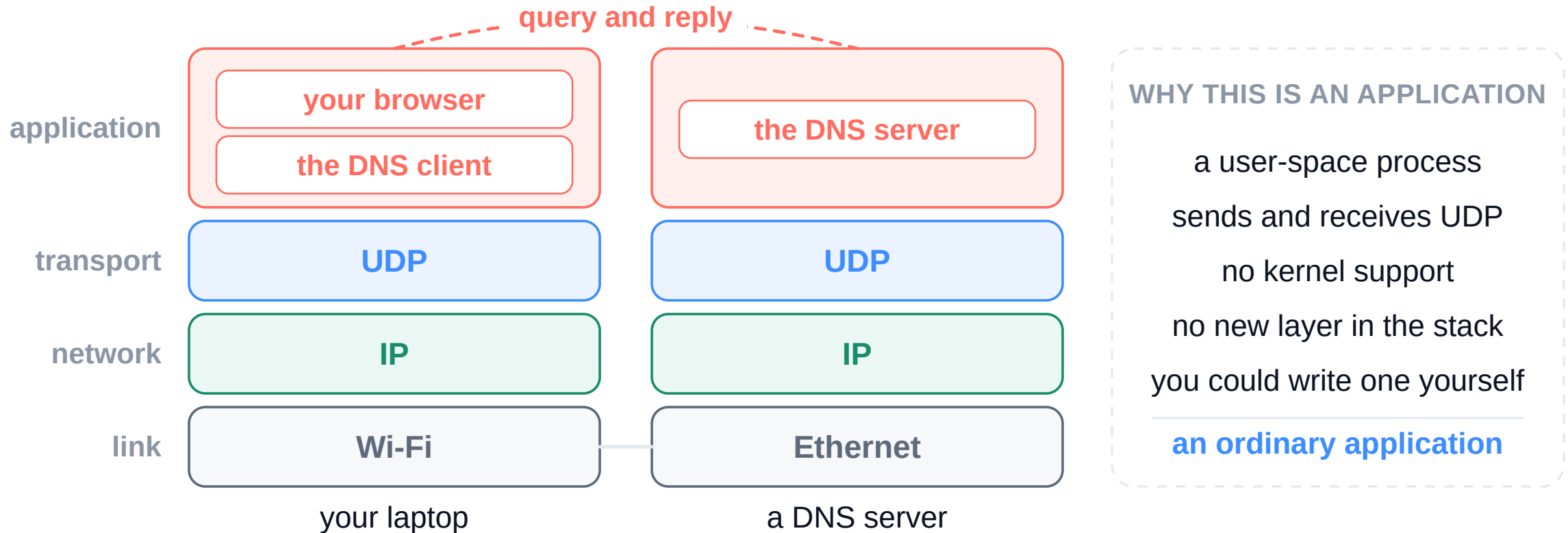
3 The tree

4 The lookup

5 Caching

6 Publishing

DNS is an application-layer protocol, like HTTP



Both ends are ordinary hosts. A DNS server is just another end point.

1 Why names

2 One server

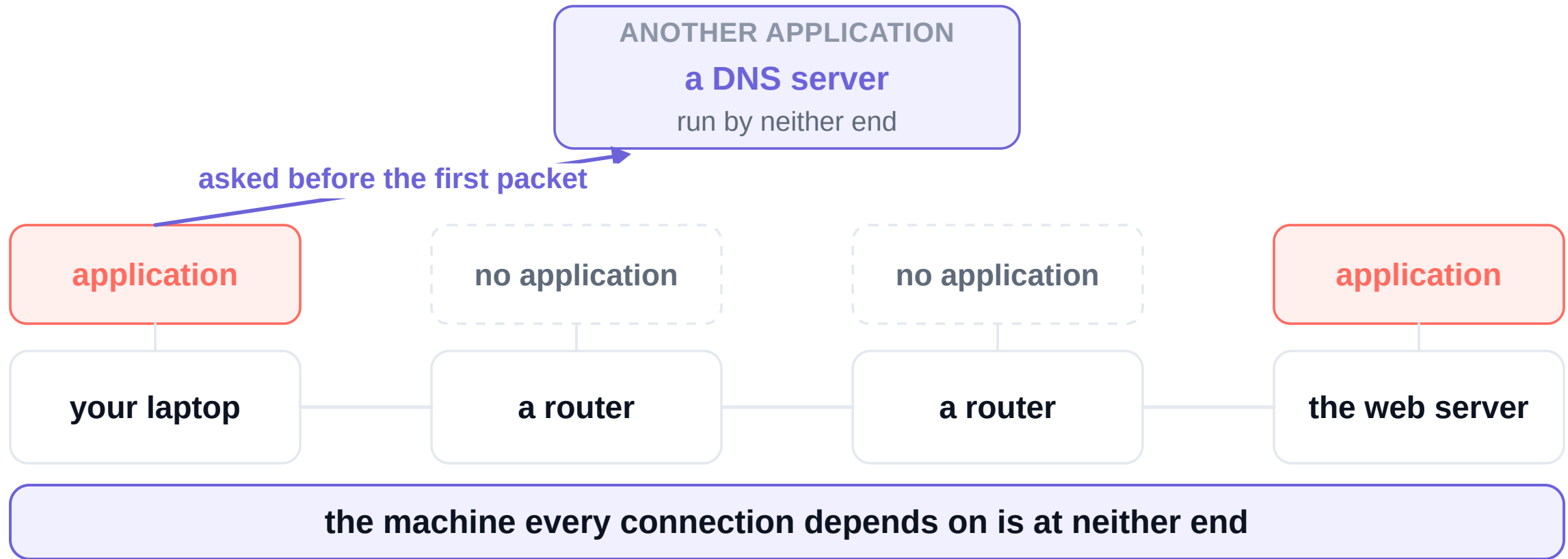
3 The tree

4 The lookup

5 Caching

6 Publishing

An application that became network infrastructure



The model puts applications at the edge. This one became infrastructure.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

You do not see it until it stops

19–20 OCTOBER 2025

one empty DNS record

a race condition in one provider's DNS automation left a regional endpoint resolving to nothing

EC2

Lambda

ECS

EKS

Fargate

STS

IAM

Redshift

15 hours of cascading failure

WHAT MAKES IT INFRASTRUCTURE

it runs before every connection

no application shows it to you

no single company owns it

thousands of independent

operators keep it running

and you notice none of it

An ordinary application, and the network cannot run without it.

- ——— Where this goes

What the rest of this lecture answers

02 What does the simplest name service do?

03 Why can one server not hold all of it?

04 How is a single name resolved?

05 Why is it fast?

06 How do I publish a name of my own?

Part 2 begins with the simplest thing that works.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

What does a name service actually have to do?

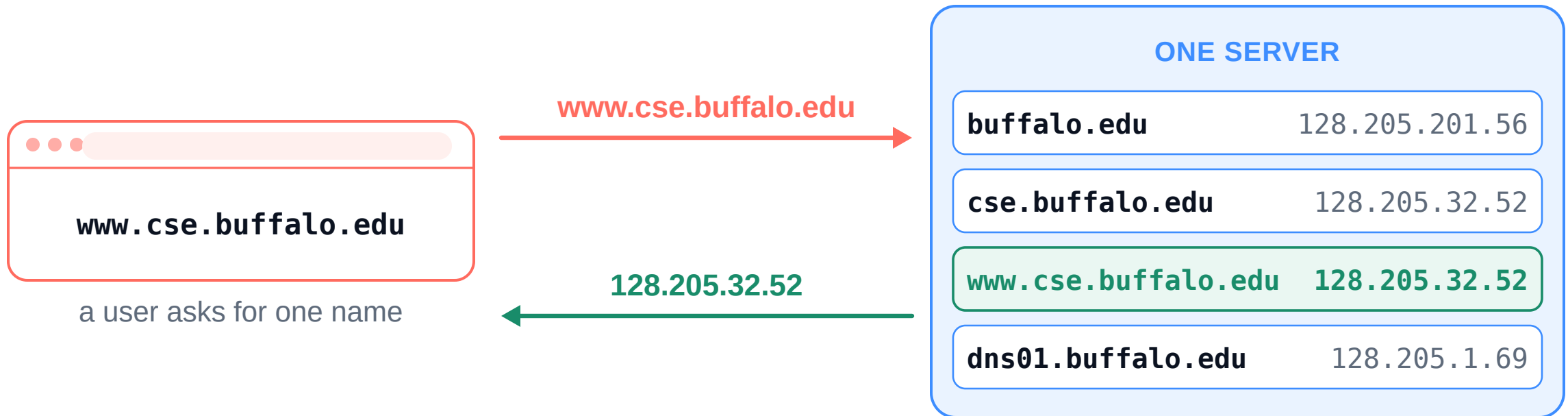
02

One server, one file

Build the smallest design that works, with nothing in it that is not needed yet. One server, one set of names, one kind of record.

• The smallest design

One server can answer for every name it is given



This is the whole exchange: the user gives a name, the server returns the address that goes with it. This really was the design of Internet naming until 1983.

Everything that follows is added to this, one problem at a time.

1 Why names

2 One server

3 The tree

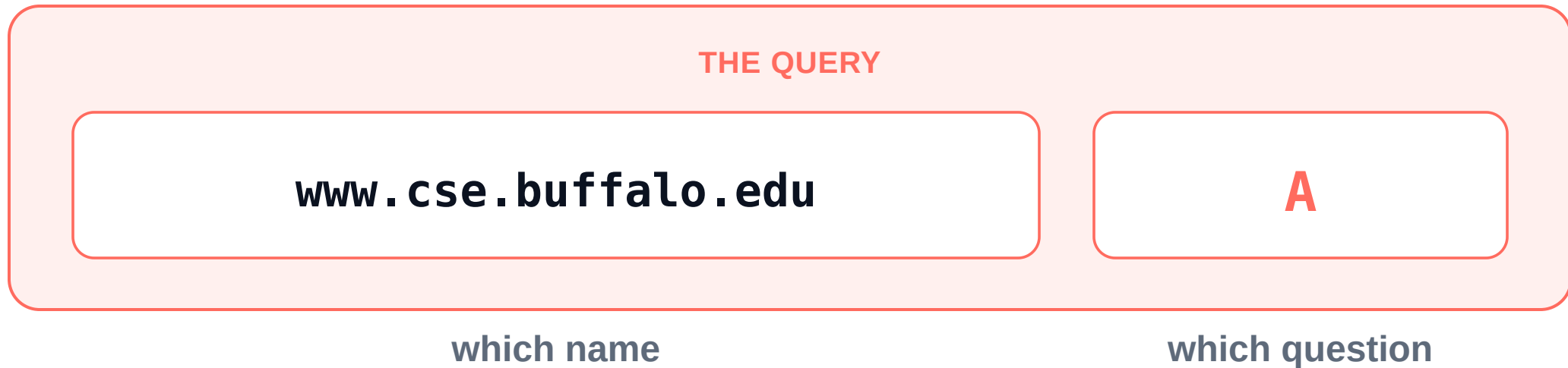
4 The lookup

5 Caching

6 Publishing

• The question

A query names one thing and asks one question



A = what is the IPv4 address of this name?

One name plus one question. That is the entire request.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

The answer is one **resource record**

<code>www.cse.buffalo.edu</code>	<code>A</code>	<code>128.205.32.52</code>	<code>21600</code>
NAME	TYPE	VALUE	TTL

NAME	the name this record is about
TYPE	which question it answers
VALUE	the answer itself
TTL	how long you may keep it — Part 5

A record is the unit DNS stores and the unit it returns.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

The pairs live in one file on that server

```
1 ; every name this server answers for
2 buffalo.edu.          IN  A  128.205.201.56
3 cse.buffalo.edu.      IN  A  128.205.32.52
4 www.cse.buffalo.edu.  IN  A  128.205.32.52
5 dns01.buffalo.edu.    IN  A  128.205.1.69
```

One record per line, one line per name. The server answers for these four names and for nothing else — a query for any other name gets no answer from it.

The server's knowledge is exactly the contents of this file.

1 Why names

2 One server

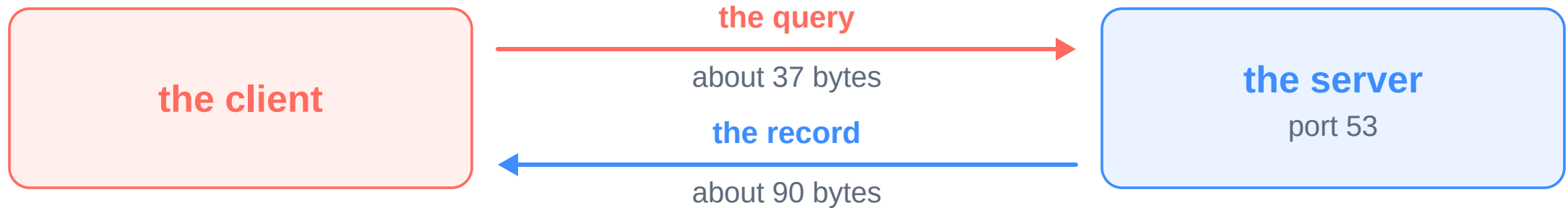
3 The tree

4 The lookup

5 Caching

6 Publishing

One datagram each way, over UDP, to port 53



WHY UDP AND NOT TCP

opening a TCP connection would cost more round trips
than the two-packet exchange it would carry

No connection, and no state kept on either side.

1 Why names

2 One server

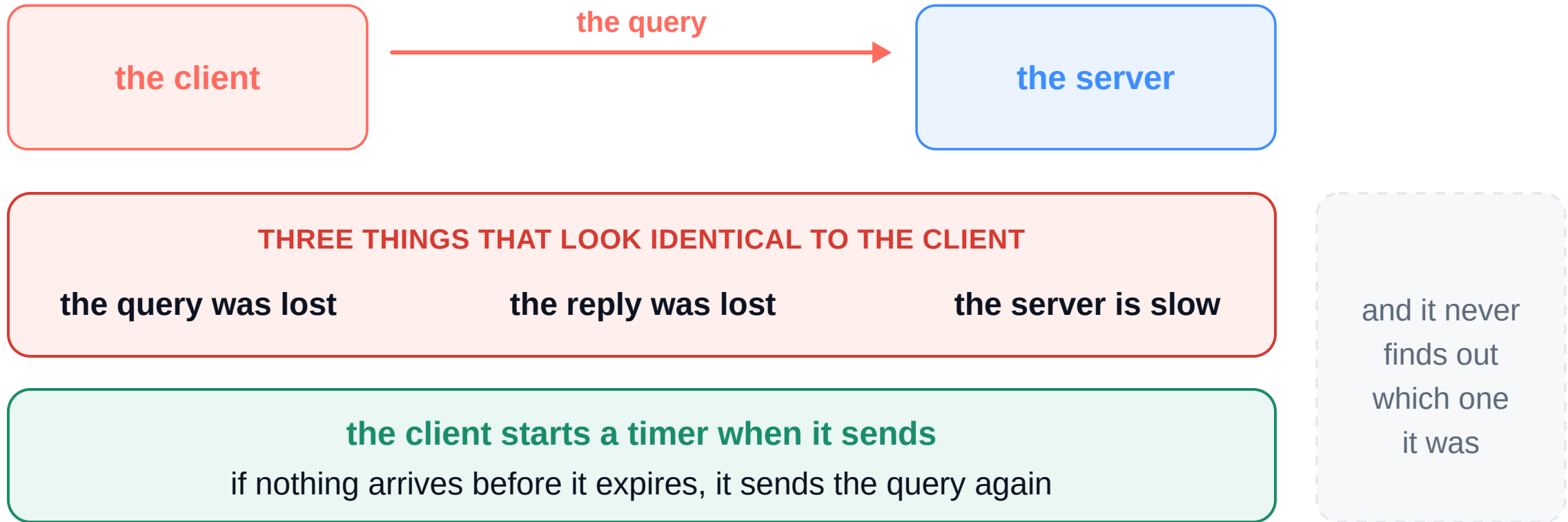
3 The tree

4 The lookup

5 Caching

6 Publishing

The client cannot tell why no answer came back



A timer and a retransmission. That is the whole recovery mechanism.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

dig shows the question and the record that answers it

```
1 $ dig www.cse.buffalo.edu A +noall +answer +stats
2 www.cse.buffalo.edu. 21600 IN A 128.205.32.52
3 ;; Query time: 31 msec
4 ;; SERVER: 192.168.1.1#53(192.168.1.1)
5 ;; MSG SIZE rcvd: 93
```

The name, the TTL, the type and the value — the four fields from two slides ago, in that order. 31 ms, 93 bytes, two packets.

You can now explain every byte of a complete name lookup.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

This is a complete name service. It has one problem.

WHAT WE HAVE

a query: a name and a type

an answer: a record

a file: the names one server holds

a transport: UDP, port 53



WHAT IS WRONG WITH IT

one server

one file

one administrator

for every name on the Internet

Part 3 is about splitting that one server into many, and who is allowed to.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

Why can one server not hold all of it?

03

The tree

Three of the four problems are about performance and are fixed by copying. The fourth is about who is allowed to write, and it is the one that decided the shape of DNS.

The Internet now has 401 million registered domains

 1983
~300 hosts
one printed page

401,600,000

registered domain names · Q2 2026

each one holds as many names below it as its owner defines

no file, and no machine, is going to hold this

The design from Part 2 is adequate at 300 rows. At four hundred million it is not, and it fails in four separate ways.

Four problems. The fourth is the decisive one.

1 Why names

2 One server

3 The tree

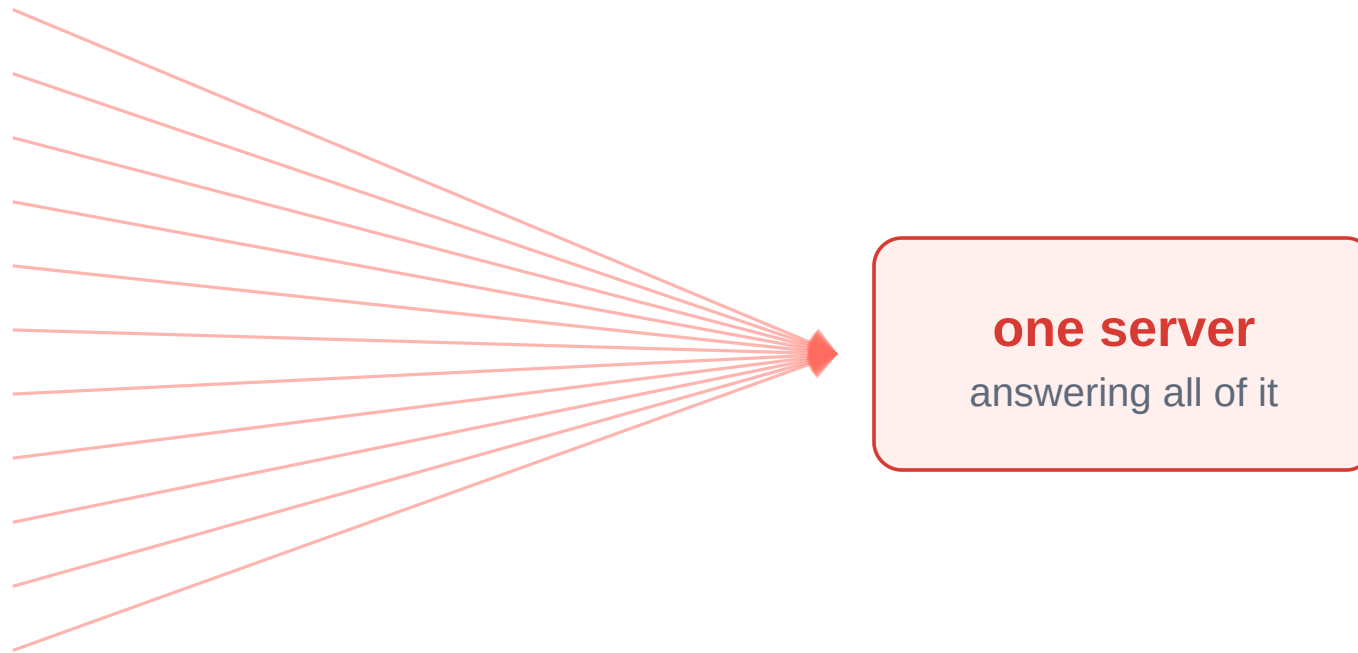
4 The lookup

5 Caching

6 Publishing

● Problem 1

One server would receive every lookup on the Internet



and it must answer
**before any other
connection can be made**

a lookup happens before every
connection, not occasionally

every device, every application, every page load

Query load. Replication solves it.

1 Why names

2 One server

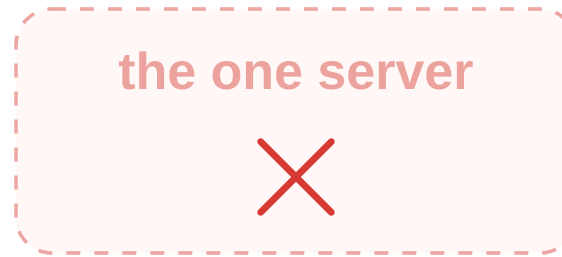
3 The tree

4 The lookup

5 Caching

6 Publishing

One server is a single point of failure



no other component has failed

every link, every router and every server is still running

and no client can reach any service

This is a different problem from the previous one. Additional capacity does not help if all of it is in one location.

Availability. Geographically separated replicas solve it.

1 Why names

2 One server

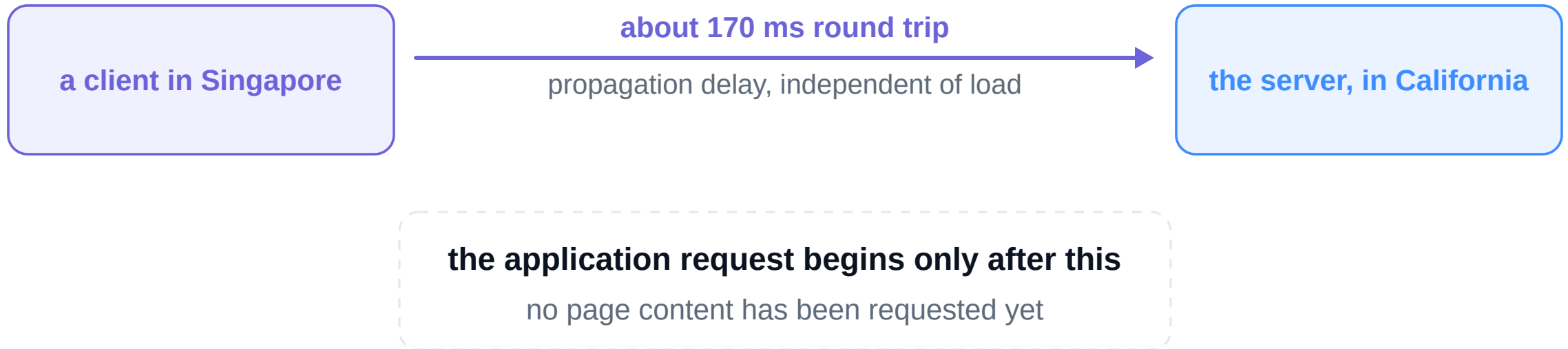
3 The tree

4 The lookup

5 Caching

6 Publishing

A distant server adds latency to every lookup



Latency. Replicas placed near clients solve it.

1 Why names

2 One server

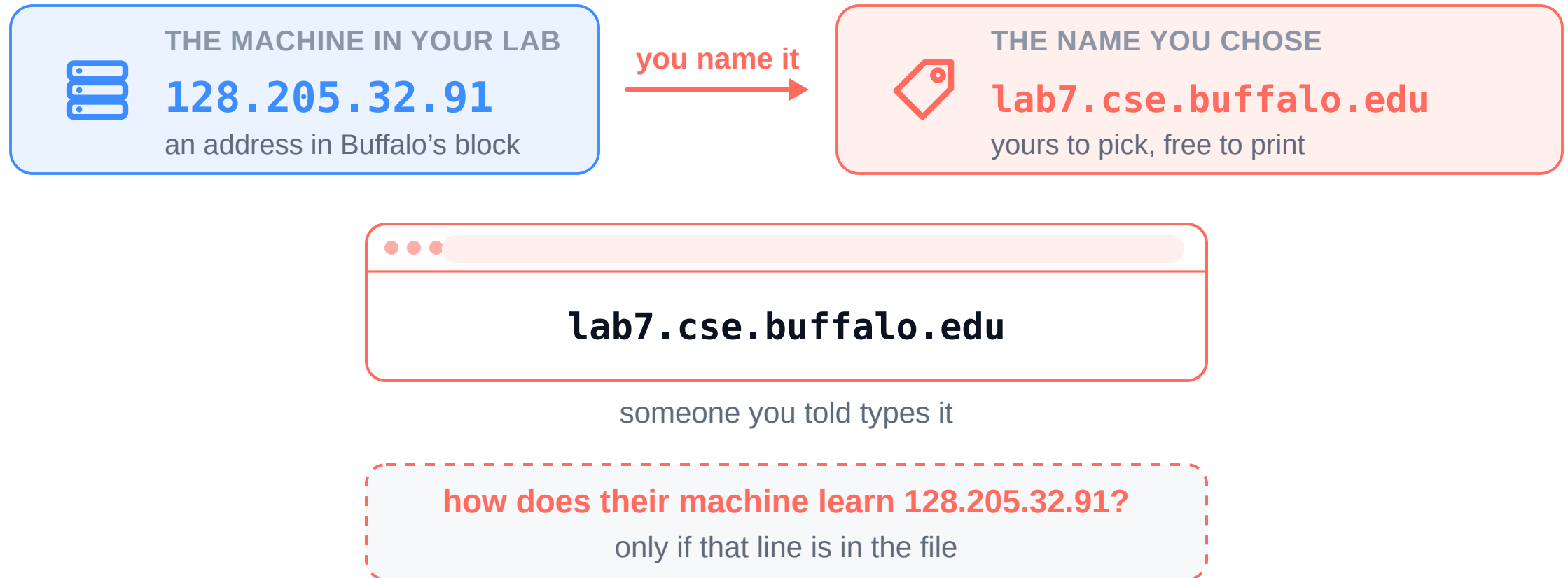
3 The tree

4 The lookup

5 Caching

6 Publishing

You want to publish lab7.cse.buffalo.edu



A name is free to choose and useless until someone writes it down.

1 Why names

2 One server

3 The tree

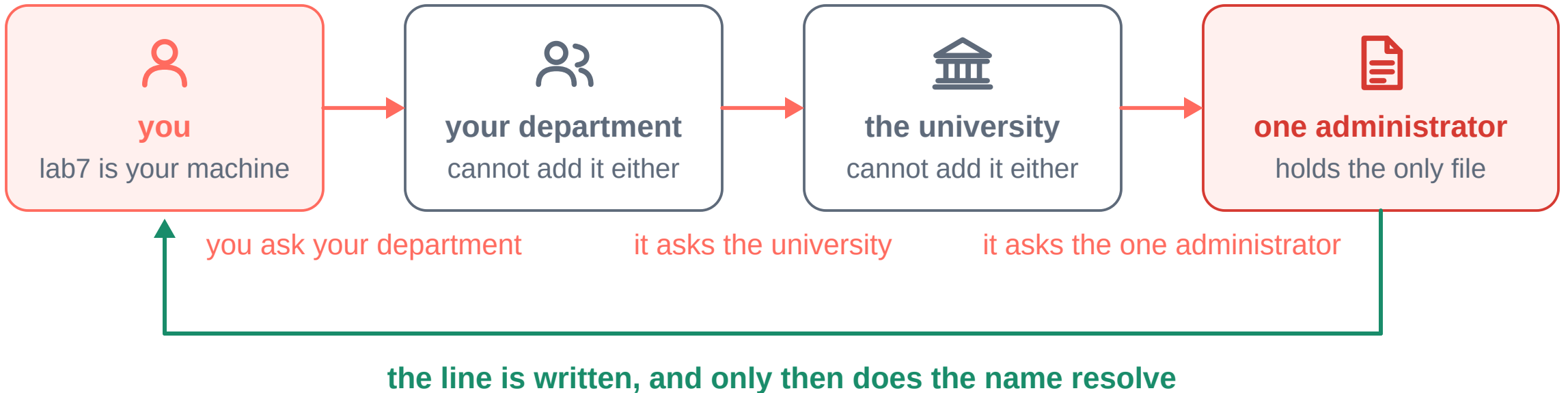
4 The lookup

5 Caching

6 Publishing

Adding one name means asking all the way up

add lab7.cse.buffalo.edu → 128.205.32.91



every name on the Internet has to pass through that last box

Nobody in the chain may add the name. Only the last box may.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

How many of these requests arrive every day



and this counts only .com and .net — 179 of the 401 million domains

a name like lab7 inside buffalo.edu is not even counted here

Verisign reported 12.7 million new .com and .net registrations during the second quarter of 2026. The per-day and per-second figures are that total divided by the length of the quarter.

Administrative control. Replication cannot solve this one.

1 Why names

2 One server

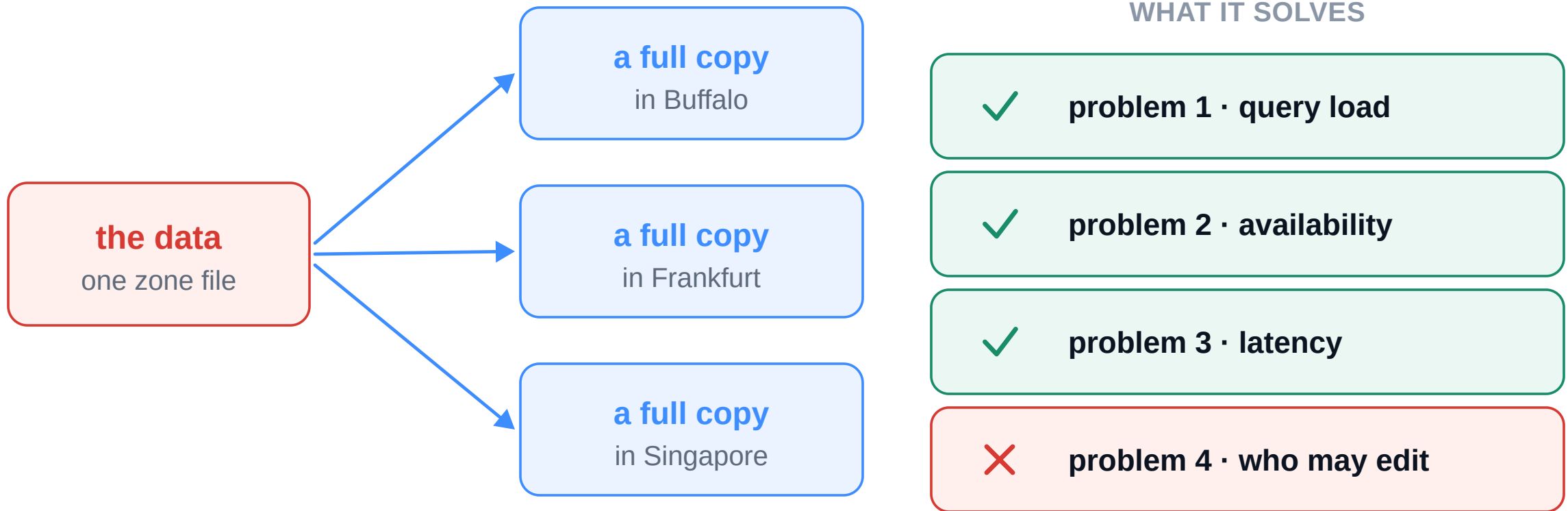
3 The tree

4 The lookup

5 Caching

6 Publishing

Copies of the data solve three of the four



Three problems gone. Every copy is still edited by the same one owner.

1 Why names

2 One server

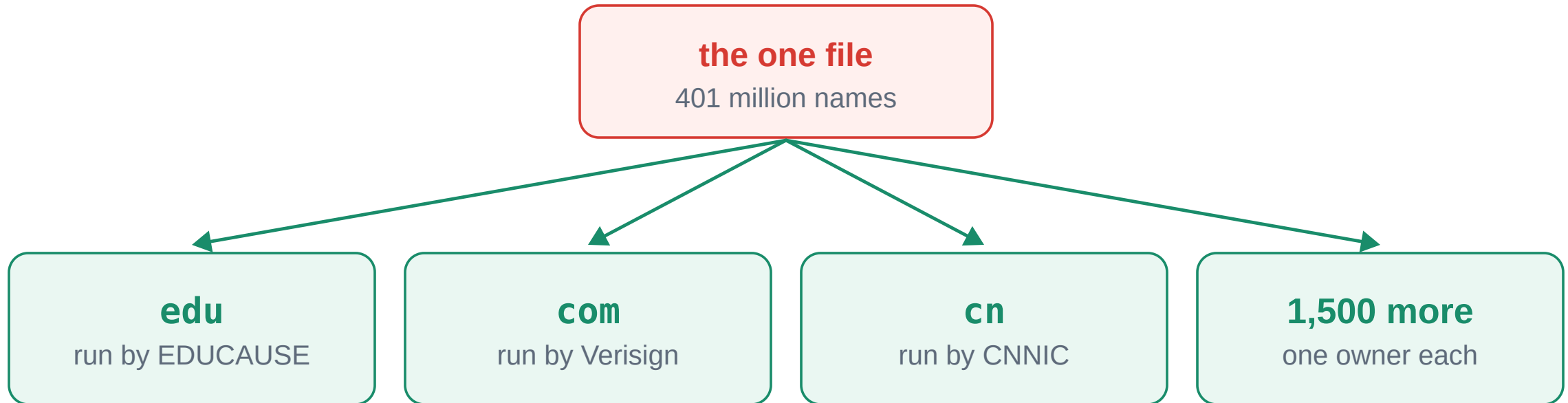
3 The tree

4 The lookup

5 Caching

6 Publishing

Splitting the data is what solves the fourth



each part has one owner, and nobody waits for the others



problem 4 is solved — authority is what got split

But each part is now a single copy again, so problems 1 to 3 come back.

1 Why names

2 One server

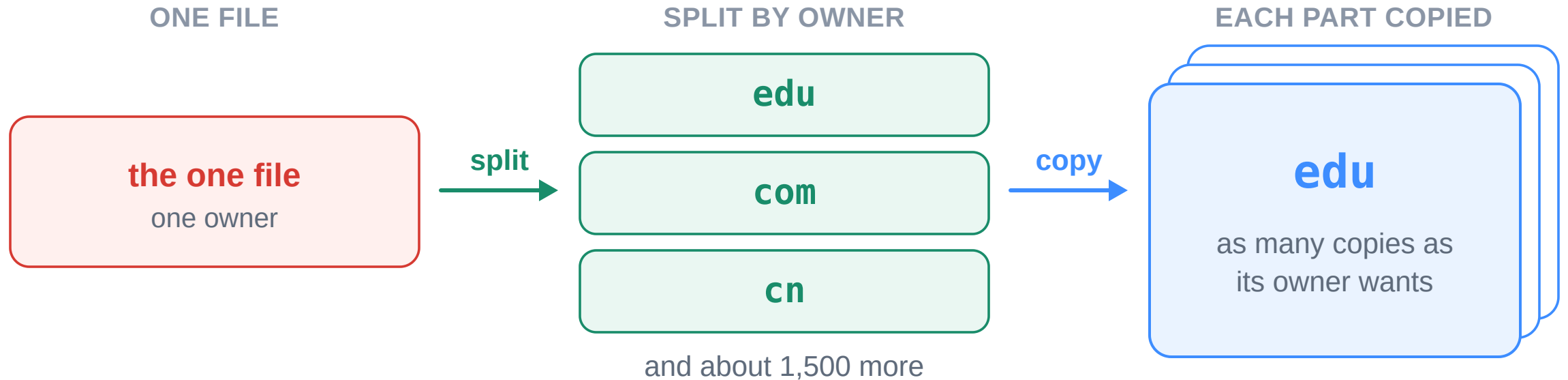
3 The tree

4 The lookup

5 Caching

6 Publishing

DNS splits first, then copies each part



IN THE OTHER ORDER

copy the whole file first, and every copy still has one owner

Partition solves problem 4. Replication then solves 1 to 3 inside each part.

1 Why names

2 One server

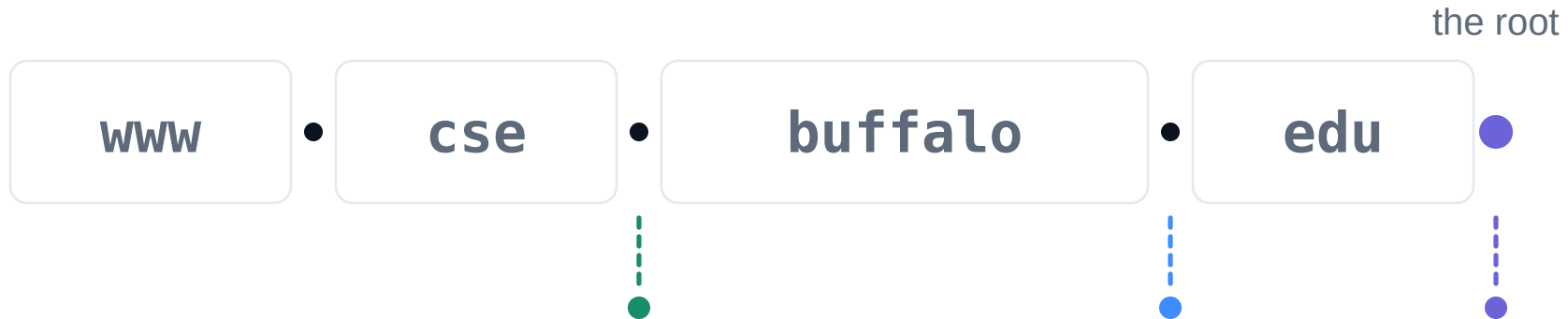
3 The tree

4 The lookup

5 Caching

6 Publishing

The dots in the name are where it can be split



the root hands edu to a registry

edu hands buffalo.edu to the university

buffalo.edu hands cse.buffalo.edu to the department

every dot is a place where authority may change hands

Nothing new had to be invented. The boundaries were already in the name.

1 Why names

2 One server

3 The tree

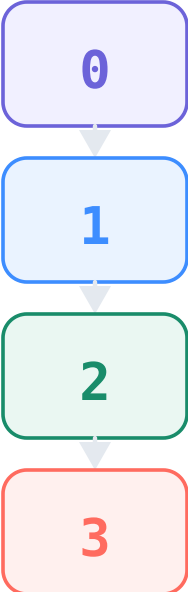
4 The lookup

5 Caching

6 Publishing

• The shape

The namespace is a tree, and every level has an owner

	LEVEL	EXAMPLE	WHO RUNS IT
	0 the root	.	ICANN, through IANA
	1 top-level domain	edu	a registry, one per TLD
	2 registered domain	buffalo.edu	whoever registered it
	3 everything below	www.cse.buffalo.edu	the owner of that domain

Each level only knows how to reach the level below it.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

How many names there are at each level

the root	1	one zone, one file
top-level domains	1,500	approximately — the root zone lists them all
registered domains	401,600,000	Q2 2026, across every TLD
names below those	?	nobody counts them, and nobody needs to

The top of the tree is tiny. Almost all of the data is at level 2 and below.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

Top-level domains come in two main kinds

COUNTRY-CODE · ccTLD

two letters, one per country

cn

de

uk

nl

about 250 in Latin letters,
and about 60 more in other scripts

GENERIC · gTLD

anyone may apply for one

com

org

xyz

app

seven of them until 2013,
about 1,200 of them now

and a dozen special ones with an eligibility rule

edu · gov · mil · int · museum · coop and arpa, used only by the protocol itself

The kind of TLD decides who may register under it, not how it is resolved.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

Some country codes are used for what the letters spell

.tv

Tuvalu

video and streaming

.ai

Anguilla

artificial intelligence

.io

British Indian Ocean Terr.

software and startups

.co

Colombia

company

.me

Montenegro

personal sites

.fm

Micronesia

radio and podcasts

.ly

Libya

word endings, bit.ly

.gg

Guernsey

gaming

.sh

Saint Helena

shell, command line

nothing in the protocol knows what the letters mean

What differs is the registry's rules, not the resolution.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

What a two-letter code is worth, and what it depends on

.ai | Anguilla

1,277,727 names registered by June 2026
\$32 million in fees in 2023, over 10% of GDP

.tv | Tuvalu

GoDaddy pays the government \$10 million a year
8.4% of all government revenue in 2019

.su | the Soviet Union

the country dissolved in December 1991
112,000 names still registered in 2026

.io | British Indian Ocean Territory

sovereignty transfer signed in May 2025
losing the ISO code starts a 5-year retirement

every one of these names works because a government's delegation still exists

A delegation is a political object as much as a technical one.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

The ten largest top-level domains

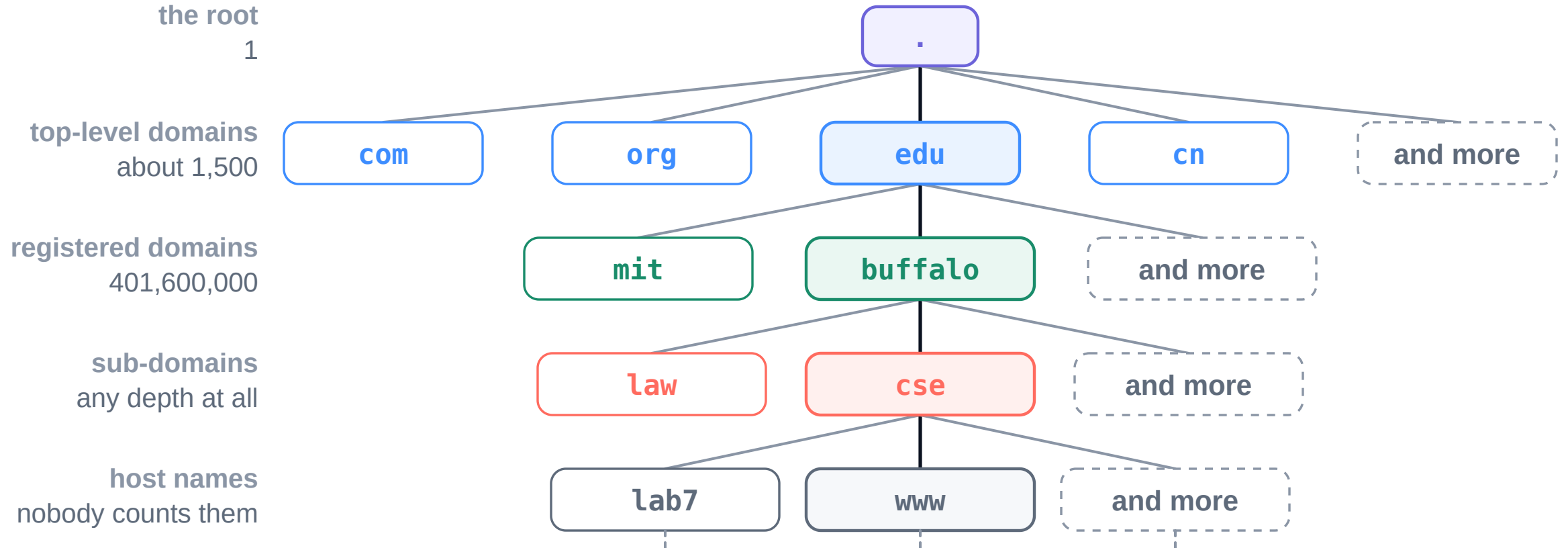


Five of the ten are country codes and five are generic. The two kinds are not separate systems; they are neighbours in the same root zone.

Size is very uneven, and the design has to work at both ends of it.

- 1 Why names
- 2 One server
- 3 The tree
- 4 The lookup
- 5 Caching
- 6 Publishing

One tree, from the root down to a single host



Only the top three levels are counted. Below them, nobody is keeping track.

1 Why names

2 One server

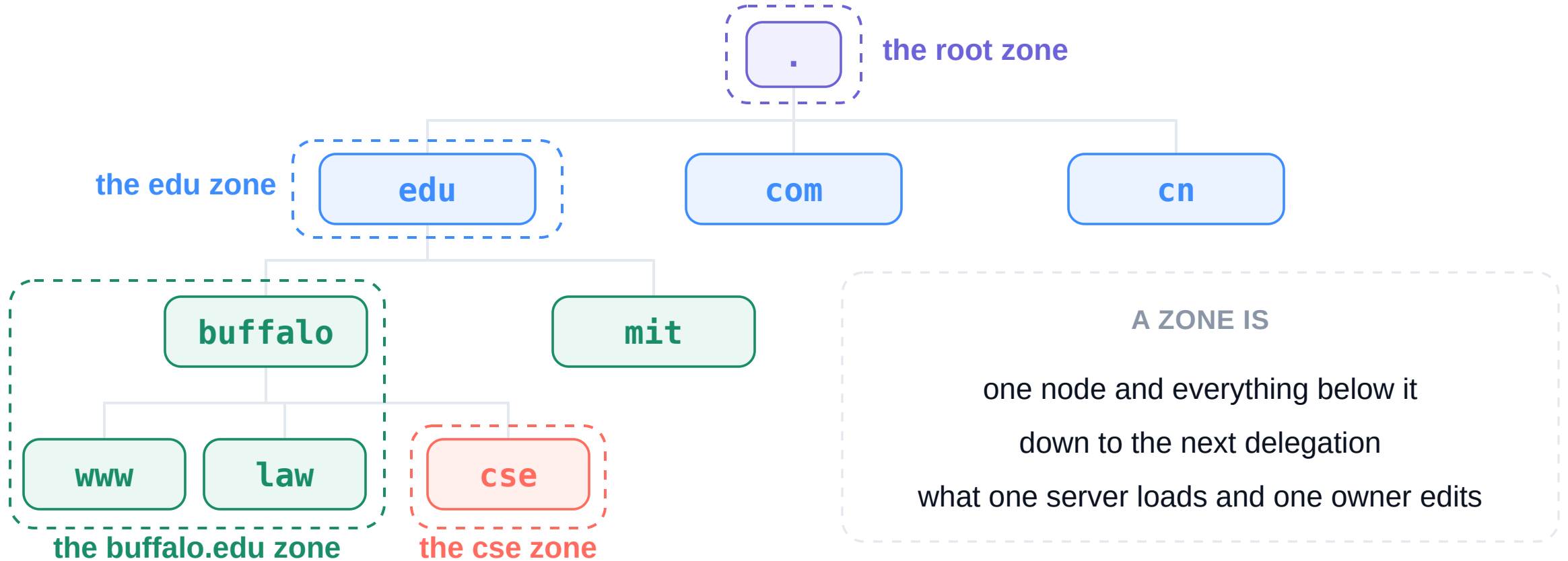
3 The tree

4 The lookup

5 Caching

6 Publishing

A zone is the part of the tree one server answers for



The single file in Part 2 was one zone. The tree is 400 million of them.

1 Why names

2 One server

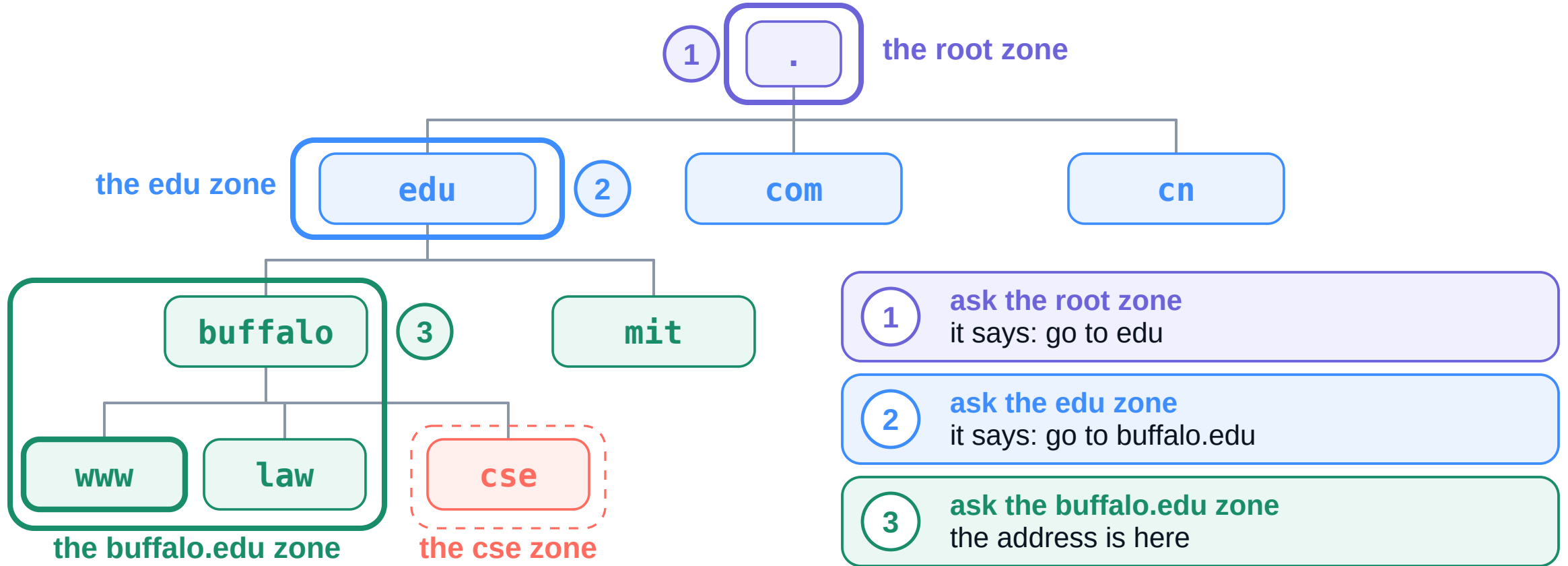
3 The tree

4 The lookup

5 Caching

6 Publishing

How the query moves from one zone to the next



Each zone knows its own names, and the name of who to ask next.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

The root has no record for buffalo.edu



the root server

its zone lists only top-level names:

com

org

edu

cn

+ 1,500 more



you ask the root for
buffalo.edu



no such record



yet on the last page it said
go to edu



so how can the root name a server
for a zone it does not hold?

Nothing in the root zone mentions buffalo. Something still points there.

1 Why names

2 One server

3 The tree

4 The lookup

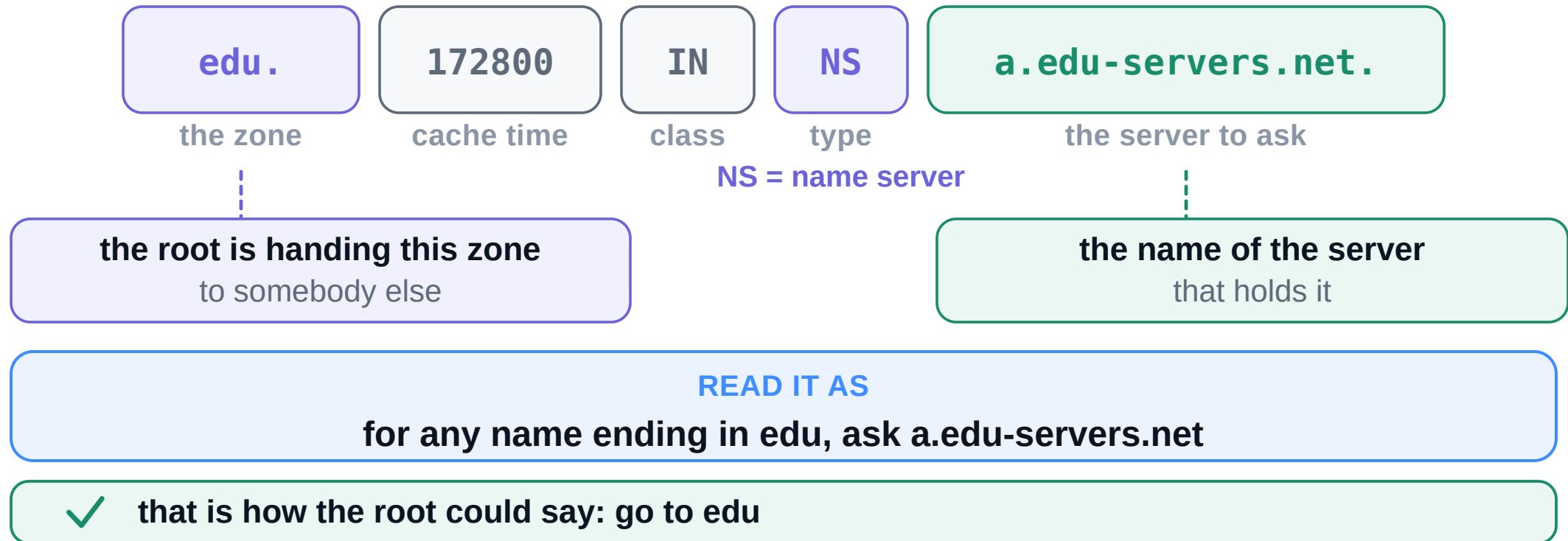
5 Caching

6 Publishing

The root zone holds one record about edu



the root zone contains this one line about edu



A parent zone stores a pointer, and nothing behind it.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

How do you reach a.edu-servers.net?



the root told you to ask
a.edu-servers.net



but that is a name
a packet needs an address

you cannot send anything to a name

SO YOU WOULD HAVE TO LOOK IT UP

ask a.edu-servers.net



what is its address?

and round again, forever

You cannot follow a referral you have no address for.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

The reply carries two records, not one

The root does not send one record. It sends two.

NS

edu. NS a.edu-servers.net.

the name of the server to ask



a name

A

a.edu-servers.net. A 192.5.6.30

and its address



an address



so you already have the IP — no second lookup

that second record is called glue

Glue is the address a parent publishes for a server it does not own.

1 Why names

2 One server

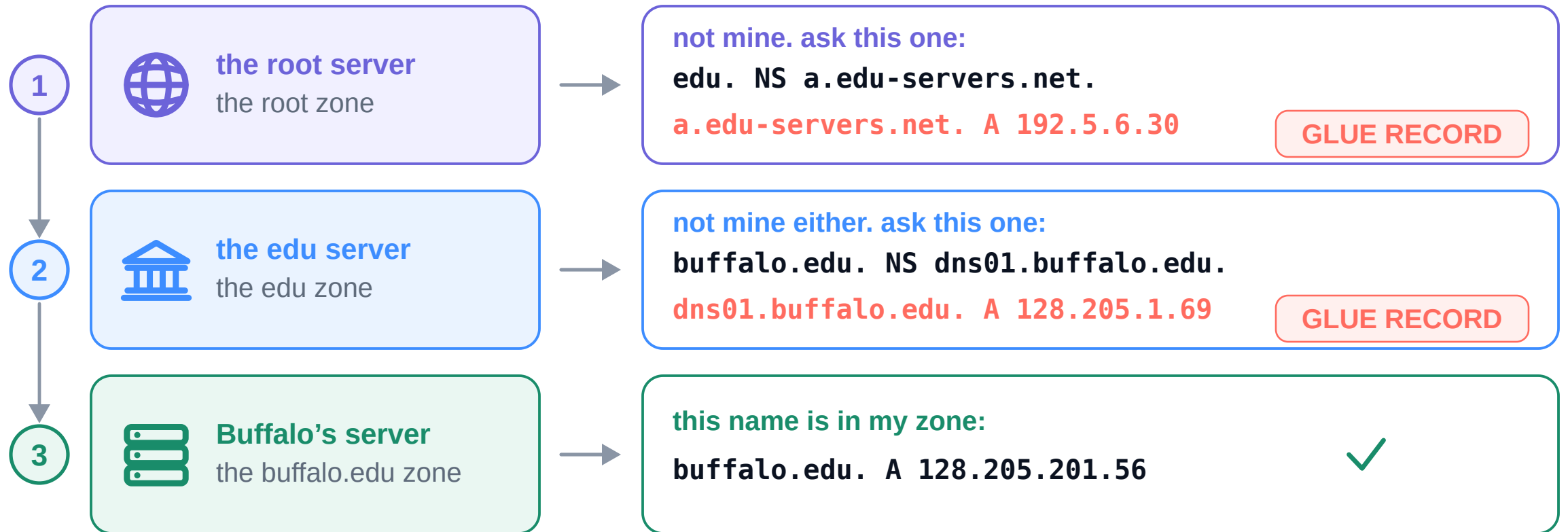
3 The tree

4 The lookup

5 Caching

6 Publishing

The same walk, with the records the servers send



Every referral is two records: the name to ask, and its address.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

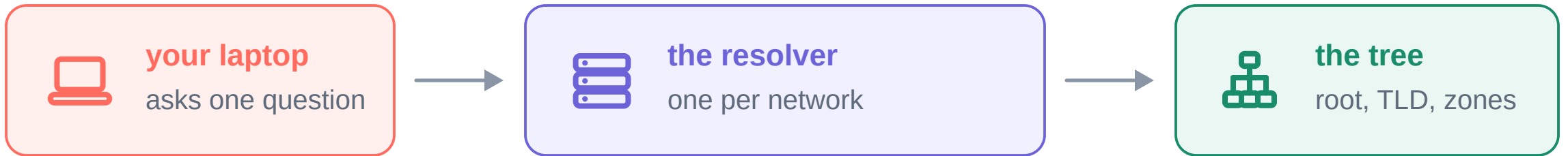
The tree is built. How is one name resolved?

04

The lookup

Your machine asks one server and waits. Every field of the DNS message you have not met yet appears in this part, at the moment it first matters.

The resolver is the program that does the walking



WHAT IT IS

a program on a server
run by your ISP or campus
or a public one: 8.8.8.8
it holds no zone of its own

WHAT IT DOES

takes your one question
walks the tree for you
sends back one answer
and remembers it for next time

Your laptop asks it one question and waits. It does everything else.

1 Why names

2 One server

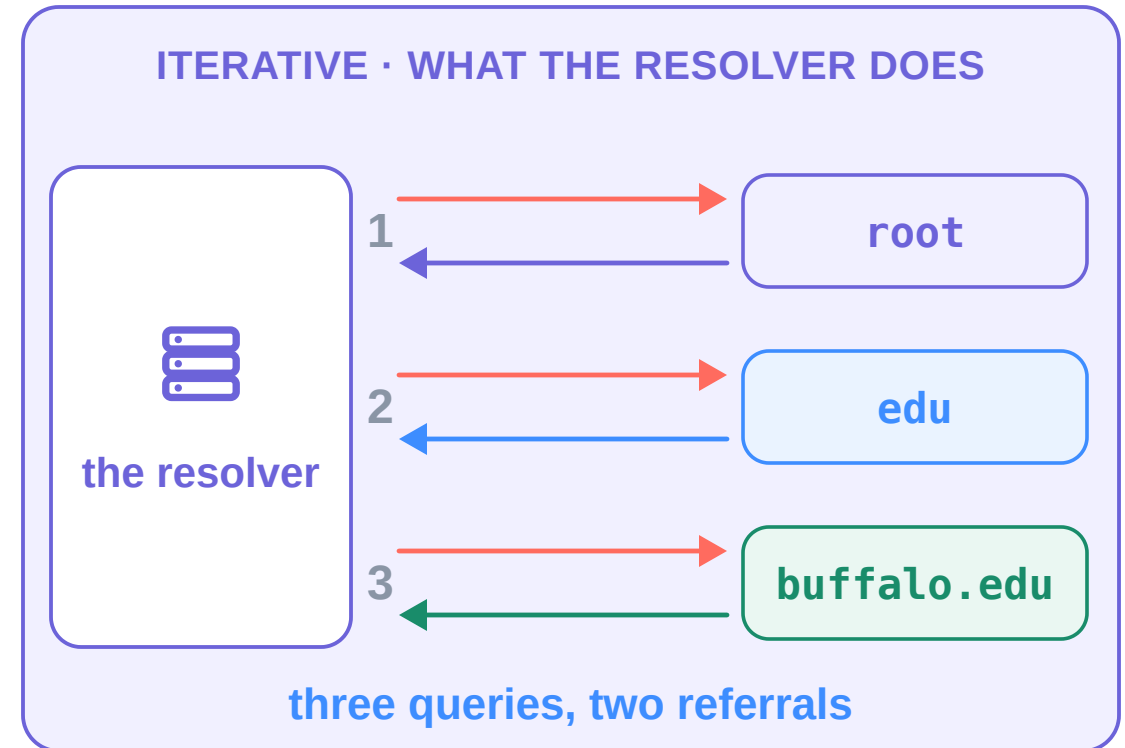
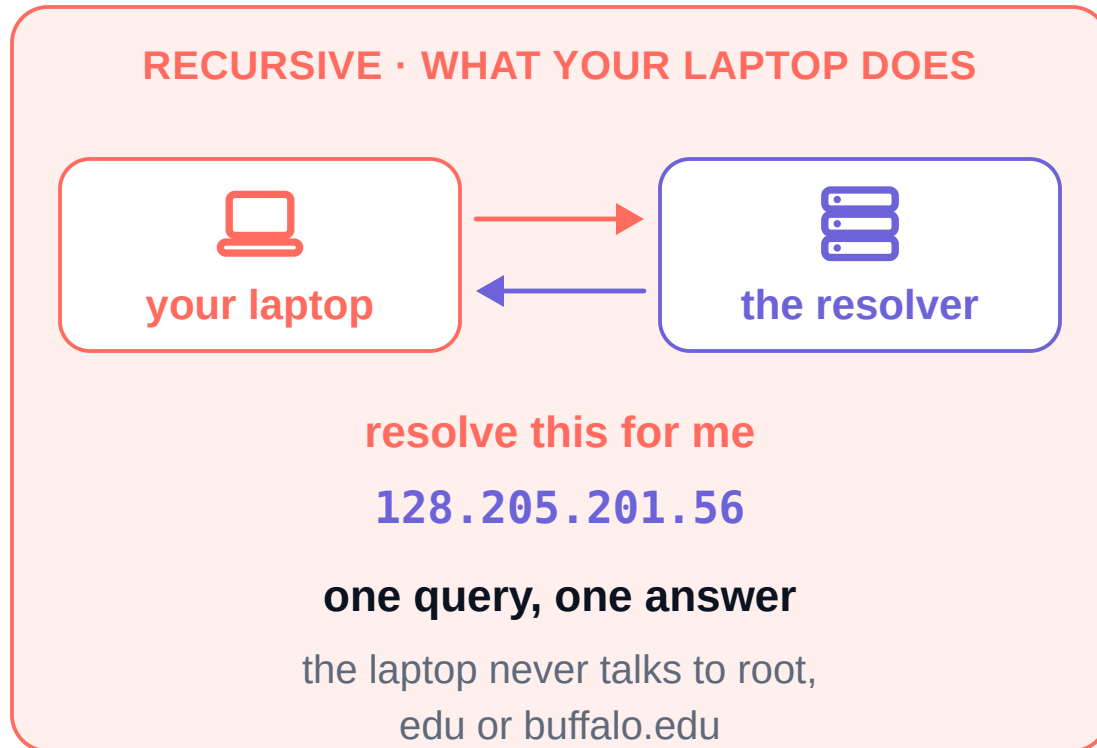
3 The tree

4 The lookup

5 Caching

6 Publishing

One recursive query becomes three iterative ones



Recursive: ask once, get the answer. Iterative: ask, get pointed onward.

1 Why names

2 One server

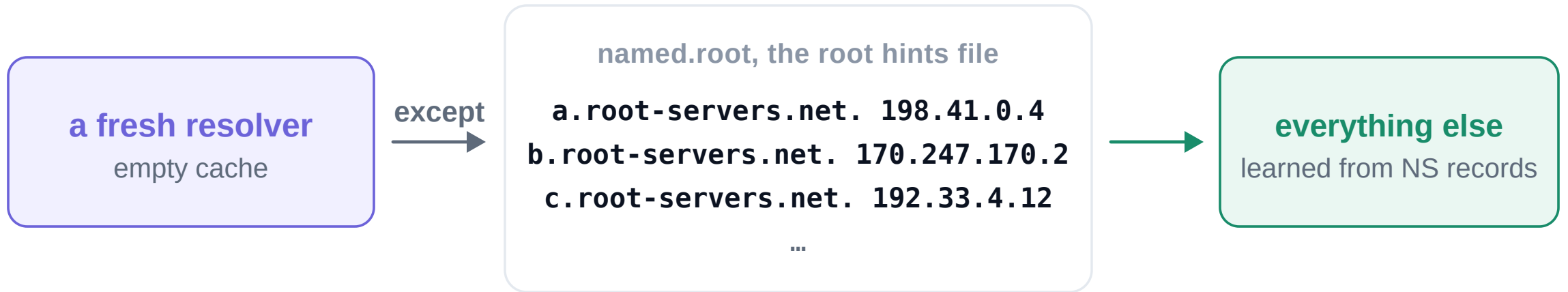
3 The tree

4 The lookup

5 Caching

6 Publishing

A resolver starts with only the root server addresses



these thirteen addresses are the only data a resolver is configured with

Every other server address is learned on the way.

1 Why names

2 One server

3 The tree

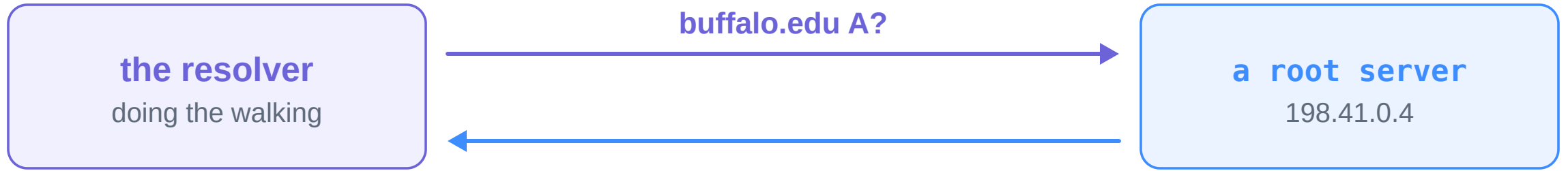
4 The lookup

5 Caching

6 Publishing

• Step 1

The root cannot answer, so it refers you instead



TWO SECTIONS YOU HAVE NOT SEEN BEFORE

AUTHORITY: edu. 172800 IN NS a.edu-servers.net.

ADDITIONAL: a.edu-servers.net. IN A 192.5.6.30

The answer section is empty. The root returns the NS record for **edu** and its address.

A referral moves the lookup one level down the tree.

1 Why names

2 One server

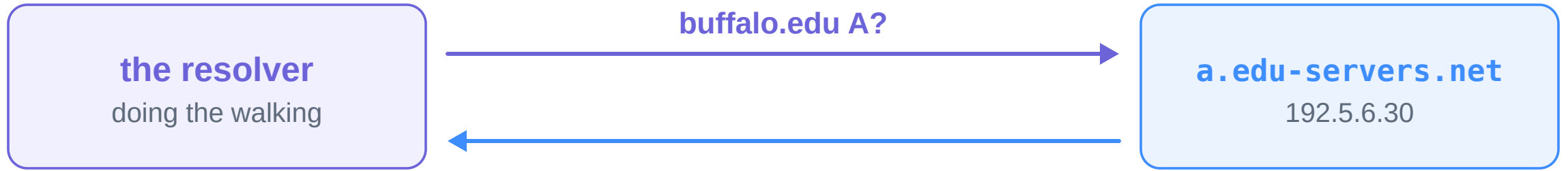
3 The tree

4 The lookup

5 Caching

6 Publishing

The .edu server refers you to buffalo.edu



THE SAME SHAPE, ONE LEVEL DOWN

AUTHORITY: buffalo.edu. 172800 IN NS dns01.buffalo.edu.

ADDITIONAL: dns01.buffalo.edu. IN A 128.205.1.69

This server holds the edu zone: delegations and almost nothing else.

Every level applies the same rule: answer, or refer.

1 Why names

2 One server

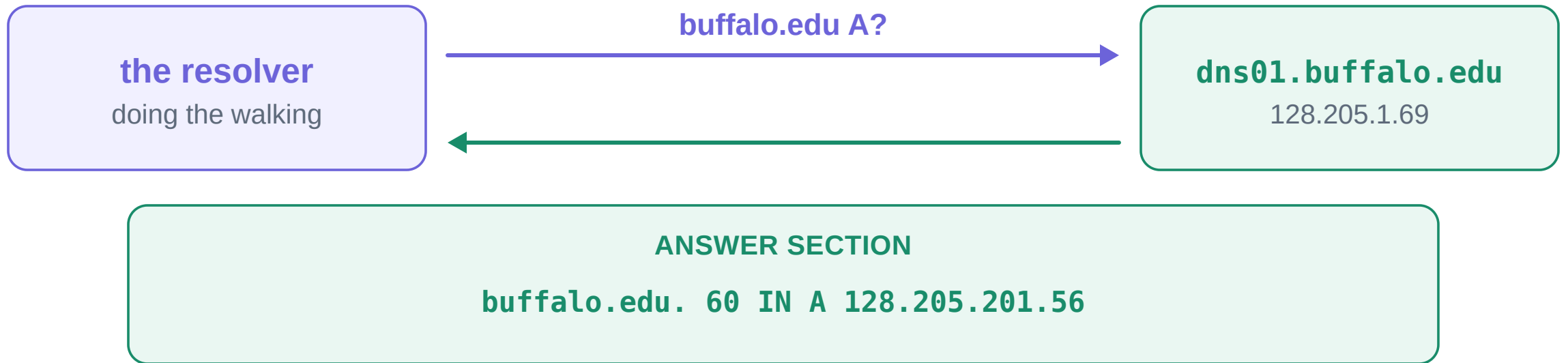
3 The tree

4 The lookup

5 Caching

6 Publishing

The server that owns the zone answers



This server holds the `buffalo.edu` zone, so it answers from its own data instead of referring you on.

Three servers, three replies, and only the last one is an answer.

1 Why names

2 One server

3 The tree

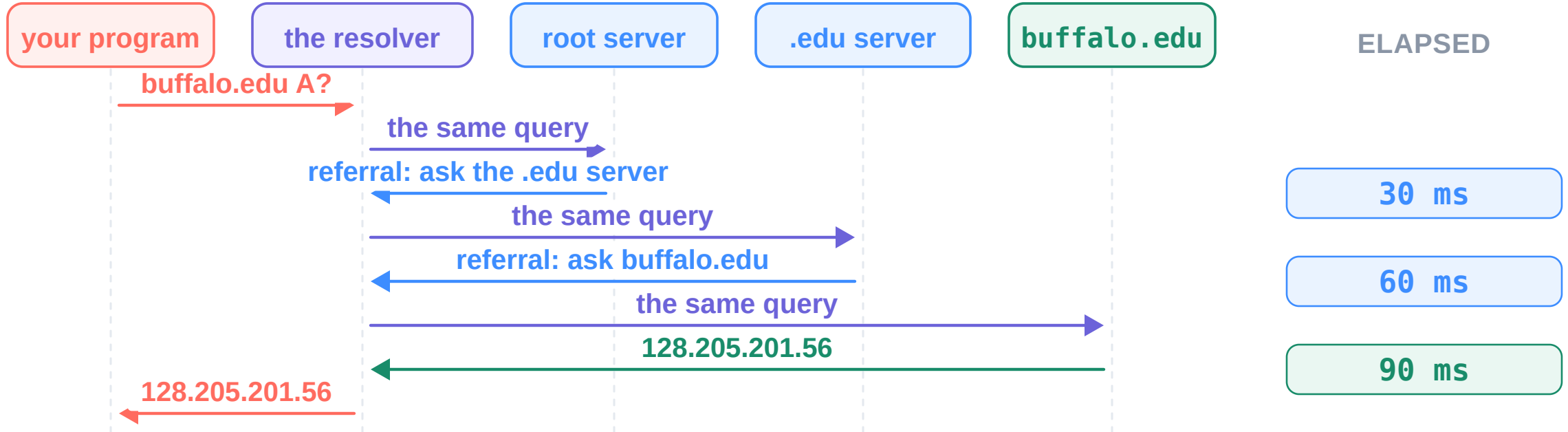
4 The lookup

5 Caching

6 Publishing

• The walk

Eight messages, four round trips, and what they cost



120 ms

before the page has started loading

four round trips, assuming 30 ms each

Nobody could afford this per page. Part 5 is why almost nobody pays it.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

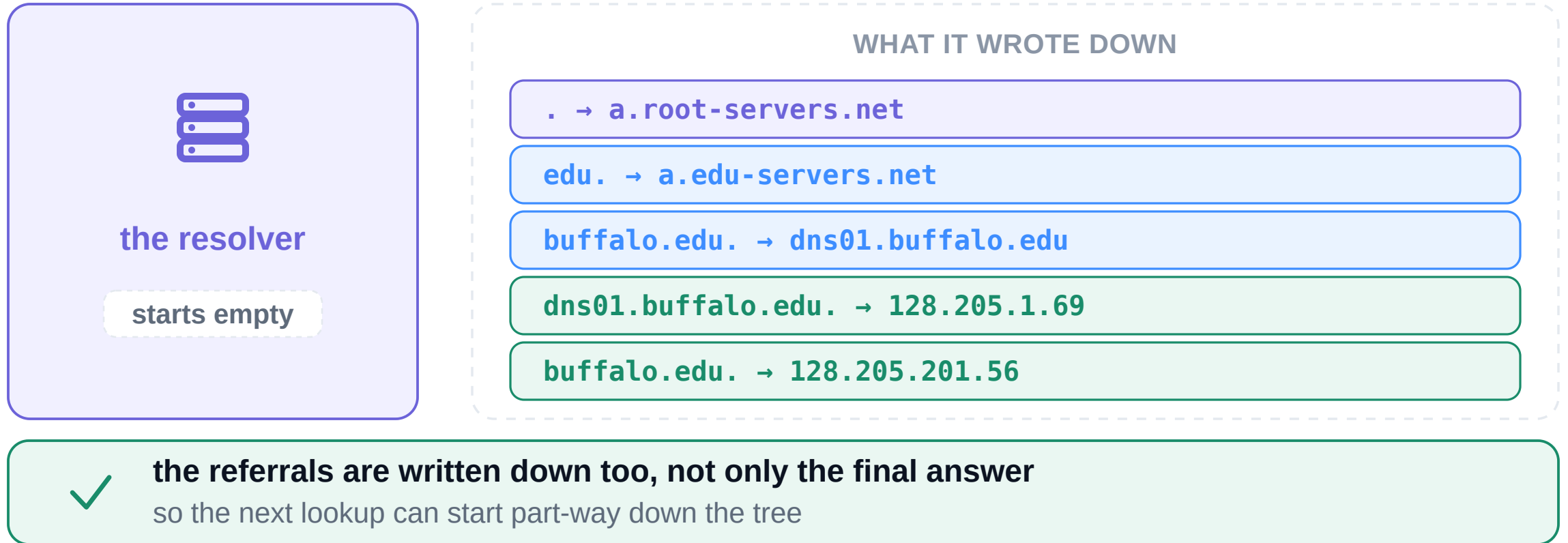
Four round trips per name. Why is the web not slow?

05

Caching

The resolver keeps what it learned, and every record carries an expiry set by whoever owns it. That one field decides how fast DNS is and how fast it can change.

The resolver writes down everything it learns



One walk fills the cache. Every later lookup starts from what is in it.

1 Why names

2 One server

3 The tree

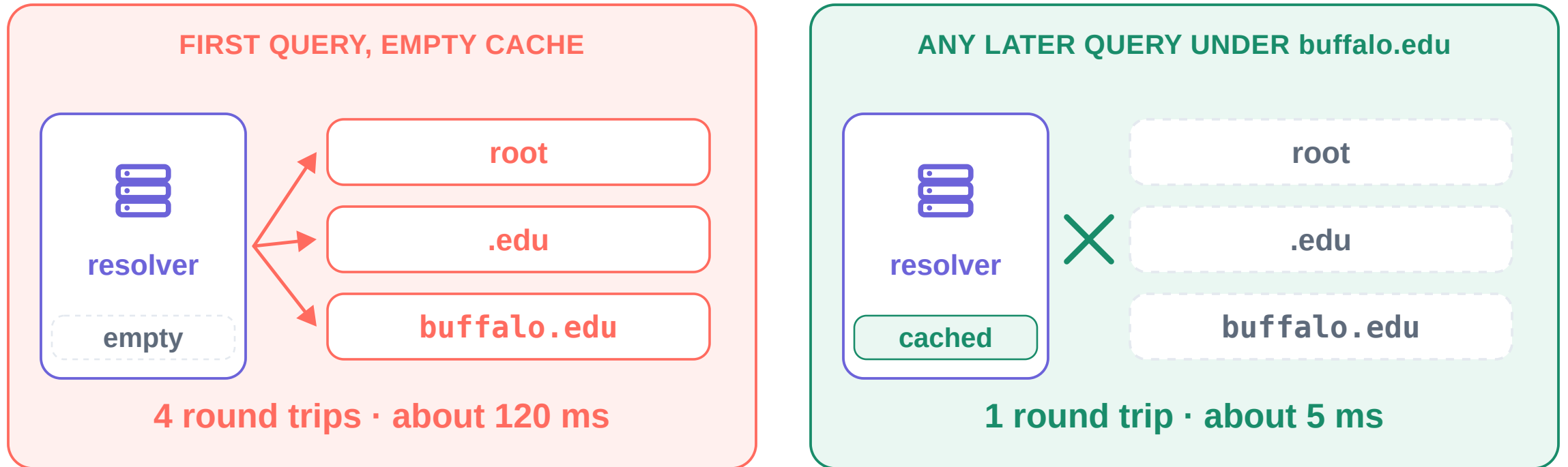
4 The lookup

5 Caching

6 Publishing

• — The payoff

A cached lookup does not touch the tree at all



✓ the referrals are cached too, so one walk pays for every later name under edu

The tree is touched only when nothing useful is cached.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

What if the address changes?



the resolver has this written down

buffalo.edu → 128.205.201.56



Buffalo moves the site to a new machine

buffalo.edu → 128.205.77.9



the resolver keeps handing out the old address



how does a cached record ever get corrected?

Nothing so far ever removes a record from the cache.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

Every record carries a TTL, and then it expires

TTL

buffalo.edu. 60 IN A 128.205.201.56

60 s — a short one



it counts down

from sixty seconds to zero



then it forgets

the record is deleted



the next query walks again

and learns the new address



the zone owner sets the TTL, so the owner sets how fast a change spreads

a record is never corrected early — it is only allowed to expire

A change takes effect when the cached copies expire, not when you make it.

1 Why names

2 One server

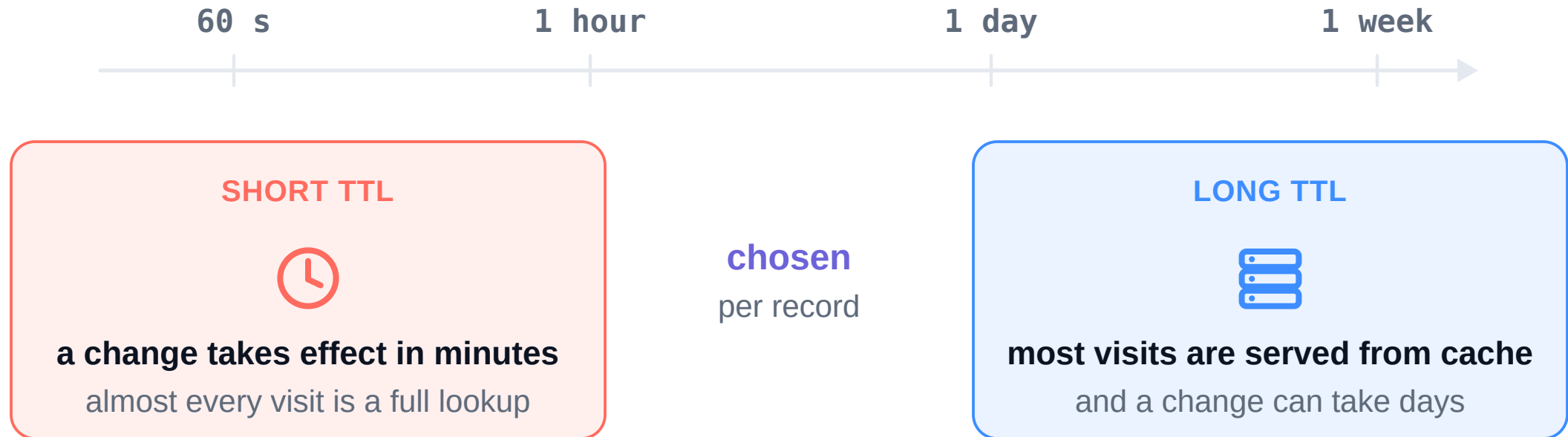
3 The tree

4 The lookup

5 Caching

6 Publishing

The TTL trades update speed against query load



Before moving a service, operators lower the TTL to 60 seconds a day in advance, make the change, and then raise it again. The extra query load is only paid during that window.

The TTL is the only control over how quickly a change takes effect.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing

How does the world learn to find your server?

06

Publishing a name

Everything so far has been reading. Publishing is a write, and part of that write happens in a zone you do not control. The record types you have not met yet appear here, one per requirement.

Three parties, and only one may edit the zone



registry

one per TLD

the only party that may edit that TLD's zone

Verisign runs .com, EDUCAUSE runs .edu, CNNIC runs .cn



registrar

a reseller

accredited by the registry to submit changes for you

GoDaddy, Namecheap, Cloudflare — you never talk to the registry



registrant

that is you

the party the name is registered to

you hold it for a term and renew it; you do not own it

You are a customer of a reseller of the one organisation that runs .com.

1 Why names

2 One server

3 The tree

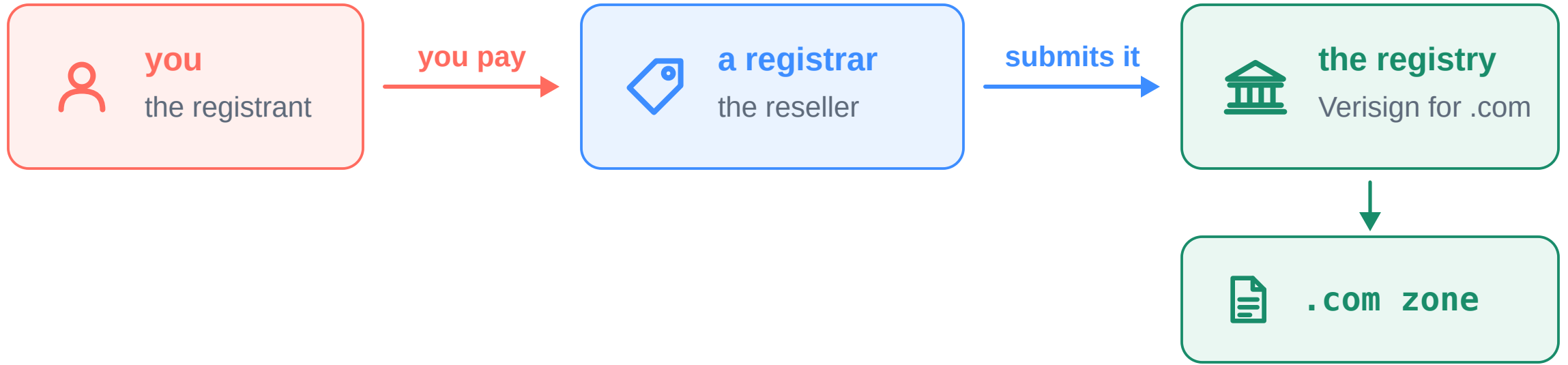
4 The lookup

5 Caching

6 Publishing

• — Step 1

What you buy, and who actually writes it down



a registration does not buy the name itself

it buys one NS line in the parent zone, renewed every year

One organisation runs .com. Everyone else is a customer of a customer.

1 Why names

2 One server

3 The tree

4 The lookup

5 Caching

6 Publishing